

BÁO CÁO PROJECT CUỐI KỲ VIETNAMESE GRAPH RAG

Hệ thống Hỏi đáp Thông minh cho
Thương mại Điện tử Tiếng Việt

Môn học: Xử Lý Ngôn Ngữ Tự Nhiên (NLP)

Nhóm trưởng : Lê Phạm Hồng Hiên – 22110059

Thành viên 1 : Dương Bùi Phương Đình – 22110049

Thành viên 2 : Phạm Xuân Huyền – 22110076

Khoa : Toán – Tin học

Giảng viên : (Tên giảng viên)

Năm học : 2025 – 2026

Mục lục

1	Giới thiệu	3
1.1	Bối cảnh và động lực	3
1.2	Phát biểu bài toán	3
1.3	Mục tiêu dự án	3
1.4	Phạm vi	4
2	Cơ sở lý thuyết	4
2.1	Tiền xử lý văn bản tiếng Việt	4
2.2	Bag of Words và TF-IDF	5
2.3	Word Embeddings: Word2Vec và GloVe	5
2.4	Mô hình Subword (BPE)	5
2.5	Mạng nơ-ron hồi quy (RNN / LSTM)	5
2.6	Transformer và BERT/PhoBERT	5
2.7	Retrieval-Augmented Generation (RAG)	6
2.8	Knowledge Graph và Graph RAG	6
3	Dataset và Tiền xử lý dữ liệu	6
3.1	Dataset sử dụng	6
3.2	Dataset chính: UIT-ViSFD	6
3.3	Dataset bổ sung: Shopee reviews	7
3.4	Phân tích nhãn aspect	7
3.5	Pipeline tiền xử lý	7
4	Kiến trúc hệ thống	8
4.1	Tổng quan Pipeline	8
4.2	Quy trình xử lý tổng thể (end-to-end)	9
4.3	Bám sát chương trình môn học (6 lecture)	10
4.4	Module 1: Tiền xử lý văn bản tiếng Việt	11
4.5	Module 2: So sánh Embedding Methods	12
4.6	Module 2b: Subword / BPE — <i>Lec03</i>	13
4.7	Module 3: Trích xuất thực thể (NER) + Gazetteer thương hiệu	13
4.8	Module 4: Knowledge Graph	14
4.9	Độ đủ của dữ liệu cho Knowledge Graph	17
4.10	Module 5: Truy xuất lai hai tầng với tái xếp hạng late-interaction	17
4.11	Module 6: Sinh câu trả lời có kiểm soát (grounded generation)	19
5	Thực nghiệm	20
5.1	Môi trường thực nghiệm	20
5.2	Giao thức đánh giá	20
5.3	Độ đo (định nghĩa hình thức)	21
5.4	Quy trình tái lập	21
5.5	Thí nghiệm 1: So sánh Embedding Methods	22
5.6	Thí nghiệm 1b: Truy vấn có <i>lexical gap</i> thật	22
5.7	Thí nghiệm 2: Trích xuất thực thể (NER)	23
5.8	Thí nghiệm 2b: Phân loại aspect bằng RNN (BiLSTM)	24
5.9	Thí nghiệm 3: Ablation chiến lược Retrieval (không rò rỉ metric)	24
5.10	Thí nghiệm 4: Demo End-to-End	25

5.11	Thí nghiệm 5: “KHÔNG dùng” vs “DỪNG” ở mức câu trả lời	26
6	Kết quả và Thảo luận	29
6.1	Đánh giá ý nghĩa thực tiễn	31
6.2	Bàn về tính đúng đắn của metrics & dữ liệu đánh giá	31
7	Triển khai LLMOps	32
7.1	Giao diện thí nghiệm (Experiment Dashboard)	33
8	Phân công công việc	36
9	Kết luận	36
9.1	Tóm tắt kết quả	36
9.2	Hạn chế	37
9.3	Hướng phát triển	38

1. Giới thiệu

1.1 Bối cảnh và động lực

Thương mại điện tử (TMDT) tại Việt Nam đạt quy mô **25 tỷ USD năm 2024** và dự báo 45 tỷ USD vào 2025 [13], với Shopee, Lazada, TikTok Shop dẫn đầu. Mỗi sản phẩm phổ biến tích lũy **hàng nghìn đánh giá** — một sản phẩm trong tập dữ liệu của chúng tôi có tới 384 review. Trước khi mua, người dùng thường có câu hỏi **theo khía cạnh cụ thể** (*pin, camera, giá, bảo hành...*) nhưng phải tự đọc và tổng hợp thủ công hàng trăm bình luận — chậm, phiền diện (chỉ đọc được vài review đầu), và dễ bị nhiễu bởi đánh giá giả.

Hệ thống tìm kiếm từ khoá truyền thống của sàn TMDT không giải quyết được vì: (i) câu hỏi ngôn ngữ tự nhiên (“pin có trâu không”) không khớp từ khoá trong review (“dùng 2 ngày mới sạc”); (ii) không tổng hợp được **xu hướng** (đa số khen hay chê pin?); (iii) tiếng Việt là ngôn ngữ đơn lập, nhiều từ lỏng/viết tắt/sai chính tả.

Retrieval-Augmented Generation (RAG) [7] kết hợp *truy xuất thông tin* và *sinh văn bản*, cho phép mô hình ngôn ngữ trả lời dựa trên **nguồn tri thức cụ thể** (review thật) thay vì “bịa” từ tham số. **Graph RAG** [8] mở rộng bằng **đồ thị tri thức** (brand → aspect → sentiment) để bổ sung ngữ cảnh thống kê — ví dụ “56% review về pin là tích cực” — mà truy xuất văn bản đơn thuần không nắm được.

1.2 Phát biểu bài toán

Dự án tập trung vào một bài toán cụ thể: **Hỏi-đáp theo khía cạnh trên đánh giá smartphone tiếng Việt** (*Aspect-based Question Answering over Vietnamese smartphone reviews*).

- **Đầu vào:** câu hỏi tiếng Việt q về một khía cạnh sản phẩm (vd. “Pin điện thoại Samsung có dùng được lâu không?”), và một kho $\mathcal{D}$ gồm N đánh giá thực tế.
- **Đầu ra:** câu trả lời ngắn a bằng tiếng Việt, **neo (grounded)** vào tập review liên quan $\mathcal{R} \subseteq \mathcal{D}$ được truy xuất, kèm thống kê cảm xúc theo khía cạnh từ Knowledge Graph.
- **Bài toán con cốt lõi** (được đánh giá định lượng): cho câu hỏi nhắm khía cạnh α (vd. BATTERY), **xếp hạng** $\mathcal{D}$ sao cho các review thực sự bàn về α nằm top đầu — đo bằng Precision@ k và MRR.

Vì sao khó? (1) *Lexical gap*: câu hỏi và review dùng từ ngữ khác nhau cho cùng khía cạnh; (2) *đa nhân*: một review thường nhắc nhiều khía cạnh (trung bình >2 aspect/review trong UIT-ViSFD); (3) *tiếng Việt*: tách từ, từ ghép, viết tắt, OOV; (4) *giảm bịa đặt*: câu trả lời phải bám dữ liệu thật, không suy diễn từ tham số LLM.

1.3 Mục tiêu dự án

1. Xây dựng hệ thống **Vietnamese Graph RAG** cho bài toán hỏi đáp về sản phẩm TMDT.
2. **So sánh** các phương pháp biểu diễn văn bản: TF-IDF, Word2Vec, GloVe, PhoBERT; có thêm subword (BPE) và mô hình tuần tự **BiLSTM**.

3. **Trích xuất thực thể** (NER) và **phân loại aspect** bằng BiLSTM từ đánh giá.
4. Xây dựng **Knowledge Graph** thể hiện quan hệ sản phẩm – thuộc tính – cảm xúc.
5. **Đánh giá** hiệu quả Graph RAG (ablation Bi-encoder / Attention / Graph).
6. **Đóng gói LLMOps**: serving API, observability, vòng lặp train→deploy mô hình.

1.4 Phạm vi

- **Domain**: Thương mại điện tử — đánh giá sản phẩm tiếng Việt (smartphone + đa sản phẩm)
- **Dataset**: (1) UIT-ViSFD [11] — 11,122 review smartphone, 10 aspect (train 7,786 dùng thực nghiệm); (2) Shopee reviews — 3,154 review đa sản phẩm (26 sản phẩm, 24 shop) có product/shop/rating. Tổng corpus truy xuất $\approx 10,9k$ review.
- **Mô hình**: TF-IDF, Word2Vec, GloVe-SVD (Lec02); Subword/BPE (Lec03); BiLSTM (Lec05); PhoBERT + Attention/MaxSim (Lec06)
- **LLM**: OpenAI API (gpt-4o-mini) cho phần Generation
- **Triển khai**: pipeline LLMops (FastAPI + Gradio + observability + CI/Docker), có vòng lặp train→deploy mô hình BiLSTM

2. Cơ sở lý thuyết

2.1 Tiền xử lý văn bản tiếng Việt

Tiếng Việt là ngôn ngữ **đơn lập** (isolating language), trong đó ranh giới từ không được xác định bằng khoảng trắng. Ví dụ: “học sinh” là một từ ghép, không phải hai từ riêng biệt.

Công cụ sử dụng: Thư viện underthesea cung cấp chức năng tách từ (word segmentation) cho tiếng Việt:

Listing 1: Tách từ tiếng Việt

```
1 from underthesea import word_tokenize
2 text = "Truong_Dai_hoc_Khoa_hoc_Tu_nhien"
3 tokens = word_tokenize(text, format="text")
4 # "Truong_Dai_hoc Khoa_hoc Tu_nhien"
```

2.2 Bag of Words và TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) đo lường mức độ quan trọng của một từ trong tài liệu so với toàn bộ tập tài liệu:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \frac{N}{1 + \text{DF}(t)}$$

Trong dự án, TF-IDF được sử dụng làm **baseline** cho việc truy xuất đánh giá sản phẩm liên quan đến câu hỏi của người dùng.

2.3 Word Embeddings: Word2Vec và GloVe

Word2Vec [5] học biểu diễn từ dựa trên *ngữ cảnh lân cận*:

- **Skip-gram**: Dự đoán từ ngữ cảnh từ từ trung tâm
- **CBOW**: Dự đoán từ trung tâm từ ngữ cảnh

GloVe [6] học embeddings dựa trên *ma trận đồng xuất hiện*, tối ưu hàm mất mát:

$$J = \sum_{i,j} f(X_{ij}) (w_i^T w_j + b_i + b_j - \log X_{ij})^2$$

2.4 Mô hình Subword (BPE)

Các embedding ở trên giả định một **từ vựng cố định**, nên khi gặp **từ hiếm hoặc ngoài từ vựng (OOV)** sẽ không có vector biểu diễn. **Byte-Pair Encoding (BPE)** [12] khắc phục bằng cách tách từ thành các **mảnh con (subword)** xuất hiện thường xuyên — ví dụ “chống_nước” → [“chống”, “_nước”]. Nhờ vậy mọi từ đều biểu diễn được qua tổ hợp subword, kể cả từ chưa từng thấy khi huấn luyện. PhoBERT sử dụng BPE trên dữ liệu tiếng Việt đã tách từ.

2.5 Mạng nơ-ron hồi quy (RNN / LSTM)

RNN xử lý dữ liệu **tuần tự** bằng trạng thái ẩn truyền qua từng bước thời gian: $h_t = f(h_{t-1}, x_t)$. **LSTM** bổ sung các cổng (input/forget/output) để giảm hiện tượng *vanishing gradient* và nắm bắt phụ thuộc xa. **BiLSTM** đọc chuỗi theo cả hai chiều rồi ghép biểu diễn, phù hợp cho bài toán phân loại câu. Trong dự án, BiLSTM được dùng để phân loại đa nhãn aspect (Mục 5.8).

2.6 Transformer và BERT/PhoBERT

Transformer [1] sử dụng cơ chế **Self-Attention** để xử lý song song toàn bộ chuỗi:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

BERT [2] mở rộng Transformer thành mô hình ngôn ngữ hai chiều pre-train quy mô lớn. **PhoBERT** [3] là phiên bản pre-train trên 20GB dữ liệu tiếng Việt, đạt state-of-the-art trên nhiều tác vụ NLP tiếng Việt; dự án dùng nó làm encoder chính.

2.7 Retrieval-Augmented Generation (RAG)

RAG [7] kết hợp hai thành phần:

1. **Retriever**: Tìm kiếm tài liệu liên quan từ knowledge base
2. **Generator**: Sinh câu trả lời dựa trên tài liệu đã tìm được

2.8 Knowledge Graph và Graph RAG

Knowledge Graph lưu trữ thông tin dưới dạng **bộ ba** (triple): (h, r, t) — head entity, relation, tail entity. Ví dụ trong TMDT:

(iPhone_15, thuộc_thương_hiệu, Apple)
 (iPhone_15, có_tính_năng, Camera_48MP)
 (Camera_48MP, được_đánh_giá, Tích_cực_85%)

Graph RAG [8] bổ sung bước **graph traversal** vào pipeline RAG, cho phép truy xuất thông tin có cấu trúc — ví dụ khi hỏi về “camera iPhone 15”, hệ thống có thể duyệt graph để tìm tất cả đánh giá liên quan đến tính năng camera.

3. Dataset và Tiền xử lý dữ liệu

3.1 Dataset sử dụng

Bảng 1: Các dataset trong dự án. Cột **Dùng?** phân biệt rõ dataset *thực sự dùng trong pipeline* với phương án chỉ tham khảo.

Dataset	Tác vụ	Kích thước	Nguồn	Dùng?
UIT-ViSFD	Aspect-Based Sentiment	11.122 comment	Shopee	✓
Shopee reviews	Review đa sản phẩm	3.154 review	GitHub (nhtlongcs)	✓
PhoNER_COVID19	NER (đã cân nhắc)	domain COVID	HuggingFace	×

*Ghi chú: PhoNER_COVID19 là **phương án domain-transfer đã cân nhắc và loại bỏ**, KHÔNG nằm trong pipeline: (i) sai domain (tin tức COVID, không phải review TMDT), và (ii) tập nhãn của nó (PATIENT, SYMPTOM, ORGANIZATION...) không có loại **BRAND/PRODUCT**. Vì thương hiệu là tập thực thể đóng, biết trước, ta dùng **gazetteer** (đúng công cụ cho tập đóng) + NER tổng quát **underthesea** thay vì NER học máy — bằng chứng ở TN2: ORG chỉ 6/800 vs gazetteer 72/800.*

3.2 Dataset chính: UIT-ViSFD

UIT-ViSFD (Vietnamese Smartphone Feedback Dataset) là bộ dữ liệu đánh giá smartphone từ sàn TMDT Việt Nam, được gán nhãn bởi con người với:

- **10 aspect categories**: SCREEN, CAMERA, BATTERY, PERFORMANCE, STORAGE, DESIGN, PRICE, GENERAL, FEATURES, SER&ACC (Service & Accessories)

- **3 sentiment polarities:** Positive, Negative, Neutral
- **11,122 comments** với annotation ở mức aspect-level

Ví dụ dữ liệu:

“Camera chụp đêm rất nét, pin dùng được 2 ngày, nhưng giá hơi cao so với cấu hình.”

→ CAMERA: Positive, BATTERY: Positive, PRICE: Negative

Listing 2: Tải và parse nhãn UIT-ViSFD (đúng định dạng thực tế)

```

1 import pandas as pd, re
2 df = pd.read_csv("data/raw/Train.csv") # cot: index, comment,
    n_star, label
3 # nhan co dang: "{CAMERA#Positive};{SER&ACC#Negative};{OTHERS};"
4 PAT = re.compile(r"\{([\w&]+)#(\w+)\}") # giu ' & ' de KHONG mat
    aspect SER&ACC
5 def parse_label(s):
6     if pd.isna(s): return []
7     return [(m.group(1), m.group(2)) for m in PAT.finditer(str(s))]
8 df["aspects"] = df["label"].apply(lambda s: [a for a, _ in
    parse_label(s)])
9 print(f"Total: {len(df)} | vi du nhan: {df['label'].iloc[0]}")

```

3.3 Dataset bổ sung: Shopee reviews

Để mở rộng ngoài domain smartphone, chúng tôi bổ sung **3.154 review Shopee đa sản phẩm** (26 sản phẩm, 24 shop) gồm các trường comment, product_name, shop_name, rating_star; sau tiền xử lý, **3.137** bản được đưa vào kho truy xuất, nên tổng index = $7.786 + 3.137 = 10.923$ review. Bộ này **không có nhãn aspect**, nên được dùng cho hai mục đích: (1) mở rộng corpus truy xuất (Graph RAG trả lời được câu hỏi đa sản phẩm), và (2) cung cấp các thực thể shop/product cho Knowledge Graph. Aspect cho review Shopee được **dự đoán bằng mô hình BiLSTM** (Mục 5.8) thay vì nhãn vàng.

3.4 Phân tích nhãn aspect

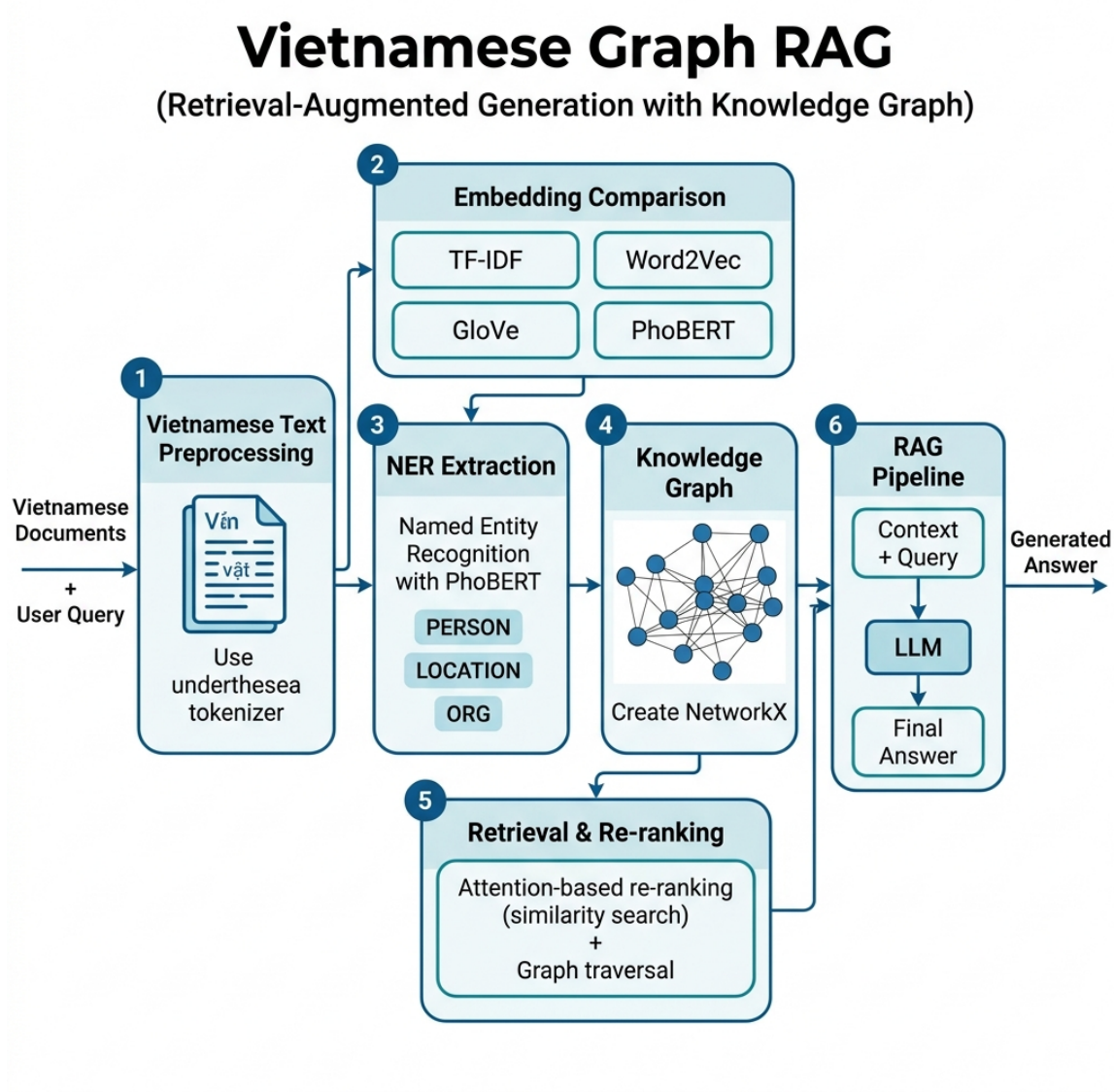
Nhãn của UIT-ViSFD có dạng chuỗi {ASPECT#Sentiment} ngăn cách bởi dấu ;. Một điểm cần lưu ý: aspect **SER&ACC** chứa ký tự &, nên biểu thức chính quy ngây thơ $\{(\w+)\#(\w+)\}$ sẽ **bỏ sót** aspect này (1.995 lần trong tập train — aspect phổ biến thứ 7). Chúng tôi sửa thành $\{([\w&]+)\#(\w+)\}$ để giữ đủ 10 aspect.

3.5 Pipeline tiền xử lý

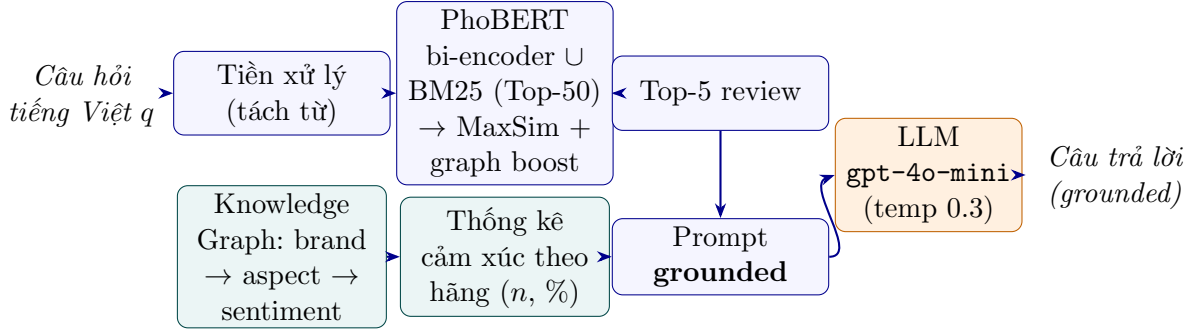
1. **Cleaning:** lowercase, loại URL, ký tự đặc biệt, chữ số, chuẩn hóa khoảng trắng.
2. **Word Segmentation:** tách từ tiếng Việt bằng underthesea.
3. **Stopword Removal:** loại từ dừng (cho TF-IDF/Word2Vec/GloVe).
4. **Lưu ý nhất quán:** PhoBERT dùng văn bản **đã tách từ nhưng giữ dấu/số** (không loại stopwords), đúng với pipeline khuyến nghị của PhoBERT; các phương pháp đếm dùng bản đã làm sạch.

4. Kiến trúc hệ thống

4.1 Tổng quan Pipeline



Hình 1: Kiến trúc tổng quan hệ thống Vietnamese Graph RAG gồm 6 module chính



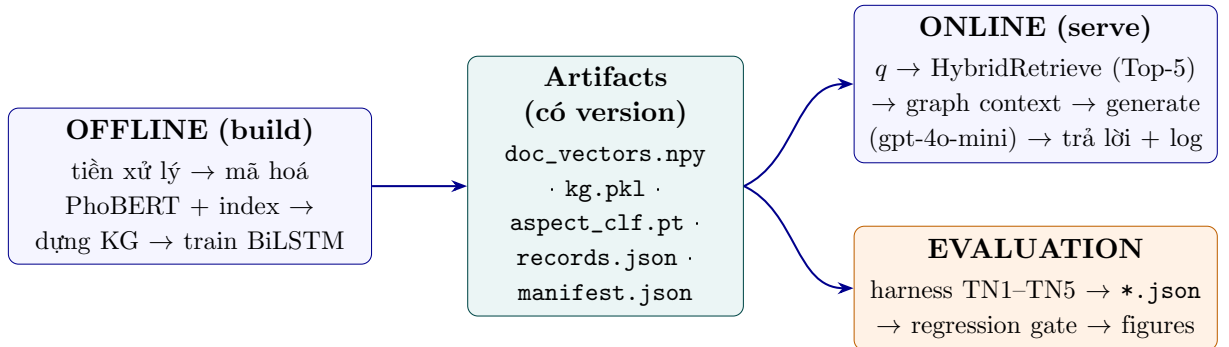
Hình 2: Luồng xử lý **Graph RAG** lúc suy luận: truy xuất lại (PhoBERT bi-encoder $\cup$ BM25 $\rightarrow$ tái xếp hạng MaxSim + graph boost) lấy Top-5 review; song song truy vấn Knowledge Graph lấy thống kê cảm xúc theo hãng. Cả hai ghép thành **prompt grounded** rồi sinh câu trả lời bằng OpenAI gpt-4o-mini.

Hệ thống gồm 6 module chính:

1. **Module 1 – Tiền xử lý:** Xử lý văn bản tiếng Việt (tách từ, loại stopwords)
2. **Module 2 – Embedding:** So sánh TF-IDF, Word2Vec, GloVe, PhoBERT
3. **Module 3 – NER:** Trích xuất thực thể bằng underthesea + gazetteer
4. **Module 4 – Knowledge Graph:** Xây dựng đồ thị tri thức (NetworkX)
5. **Module 5 – Retrieval:** Embedding + Graph boost + Attention (MaxSim) re-ranking
6. **Module 6 – Generation:** Tích hợp LLM API (OpenAI)

4.2 Quy trình xử lý tổng thể (end-to-end)

Trước khi đi vào từng module, phần này trình bày **quy trình xử lý end-to-end**: dữ liệu chảy qua các bước nào và sinh ra artifact gì. Hệ thống tách **hai pha** — *offline* (dựng artifact **một lần**, có version) và *online* (nạp lại artifact để phục vụ truy vấn và đánh giá), đúng nguyên tắc LLMOps (Mục 7).



Hình 3: Quy trình end-to-end: pha **offline** dựng artifact có version; pha **online** và **đánh giá** cùng nạp lại artifact đó (một nguồn sự thật).

Bảng 2: Quy trình 9 bước: Hành động → Output (artifact). Lệnh tái lập từng bước ở Mục 5.4.

Bước	Pha	Hành động	Output
1	Offline	Load UIT-ViSFD + Shopee, parse nhân, gộp corpus 10.923	<code>records.json</code>
2	Offline	Tiền xử lý (2 pipeline: đếm / PhoBERT)	—
3	Offline	Mã hoá PhoBERT mean-pool + dựng index có version	<code>doc_vectors.npy</code> , <code>manifest.json</code>
4	Offline	Dựng KG (brand→aspect→sentiment; shop→product)	<code>kg.pkl</code>
5	Offline	Train BiLSTM aspect; gán aspect cho Shopee làm giàu KG	<code>aspect_clf.pt</code>
6	Online	$q \rightarrow$ HybridRetrieve hai tầng (Top-5 review)	review liên quan
7	Online	Truy vấn KG theo hãng → thống kê cảm xúc ($n, \%$)	graph context
8	Online	Prompt grounded → LLM → câu trả lời + log chi phí	answer, <code>queries.jsonl</code>
9	Eval	Harness TN1–TN5 → regression gate → figures	<code>metrics/*.json</code>

4.3 Bám sát chương trình môn học (6 lecture)

Các thành phần của hệ thống ánh xạ trực tiếp tới 6 bài giảng của môn:

Lecture	Nội dung	Áp dụng trong project
Lec01 – Tiền xử lý & Tách từ	cleaning, word segmentation, stopword	Module 1: <code>preprocess_vietnamese</code> (dùng chung train↔inference)
Lec02 – Word Representation	one-hot, TF-IDF, Word2Vec, GloVe	So sánh TF-IDF / Word2Vec / GloVe-SVD
Lec03 – Subword Models	BPE, subword	Tokenize BPE (PhoBERT) cho từ hiếm/OOV
Lec04 – Compositional Semantics	Document→Sentence→Phrase→Word	Graph: brand→aspect→sentiment, shop→product
Lec05 – Recurrent Neural Networks	RNN/LSTM, dữ liệu tuần tự	BiLSTM phân loại đa nhãn aspect (micro-F1)
Lec06 – Attention & Transformer	attention, encoder–decoder	PhoBERT + tái xếp hạng MaxSim (late-interaction)

Bảng 3: Ánh xạ thành phần hệ thống ↔ 6 lecture của môn.

4.4 Module 1: Tiền xử lý văn bản tiếng Việt

Tiếng Việt là ngôn ngữ đơn lập nên ranh giới từ không trùng khoảng trắng; chất lượng mọi biểu diễn downstream phụ thuộc bước tách từ. Hệ thống dùng **hai pipeline tiền xử lý song song**, chọn theo bản chất mô hình tiêu thụ đầu ra.

(a) Pipeline “đếm” (TF-IDF / Word2Vec / GloVe-SVD / BiLSTM) — hàm `preprocess_vietnamese` áp dụng tuần tự:

- Hạ chữ thường + cắt khoảng trắng thừa.
- Loại URL (`http\S+|www\S+`).
- Loại ký tự không phải chữ/số (`[^\w\s]`) — bỏ dấu câu, emoji.
- Chỉ loại số đứng riêng** (`\b\d+\b`); *giữ* token chữ-số mang nghĩa cấu hình (64gb, 120hz, 5g). Đây là quyết định thiết kế: xóa sạch chữ số sẽ làm mất tín hiệu domain điện thoại.
- Tách từ bằng `underthesea`.
- Loại **stopword** (tập 47 hư từ: *và, của, là, có, cho, được, trong...*) và token độ dài ≤ 1 .

(b) Pipeline “ngữ cảnh” (PhoBERT) — *chỉ* tách từ (`word_tokenize`), **giữ nguyên dấu, số và stopwords**, đúng khuyến nghị của PhoBERT (pre-train trên văn bản tách từ có dấu). Loại stopwords/số ở đây sẽ phá vỡ ngữ cảnh mà self-attention khai thác.

Bảng 4: Ví dụ hai pipeline trên cùng một đầu vào.

Đầu vào	"Pin iPhone 13 trâu, sạc nhanh 120Hz :)) http://shp.ee/x"	
(a) Dếm	pin iphone trâu sạc nhanh 120hz <i>URL/emoji/"13"; giữ "120hz"</i>	(bỏ
(b) PhoBERT	Pin iPhone 13 trâu , sạc nhanh 120Hz <i>từ)</i>	(chỉ tách

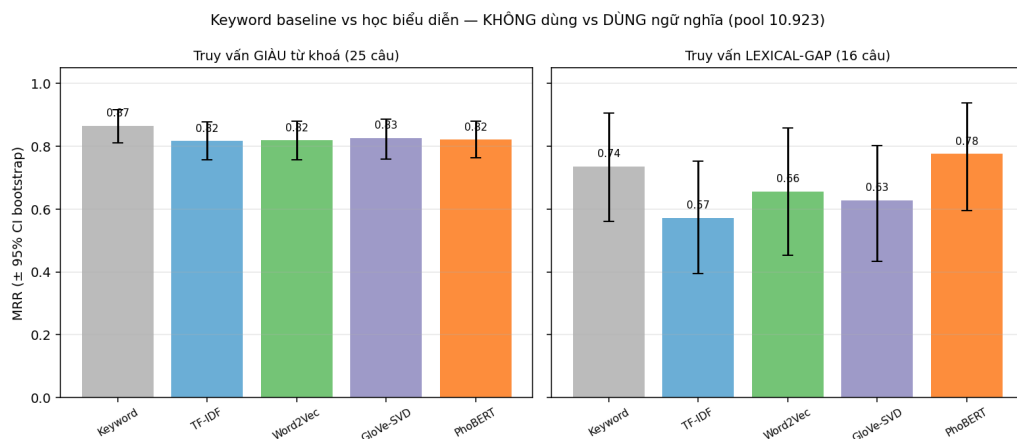
Nhất quán train↔inference: cùng một hàm `preprocess_vietnamese` được gọi ở cả lúc huấn luyện BiLSTM lẫn lúc suy luận và gán nhãn Shopee. Nếu hai pha lệch nhau, embedding-lookup lúc deploy sẽ trượt OOV hàng loạt — đây là lỗi triển khai phổ biến mà thiết kế “một nguồn sự thật” này tránh được.

4.5 Module 2: So sánh Embedding Methods

Đây là module quan trọng nhất, thể hiện kiến thức NLP cốt lõi. Chúng tôi so sánh 4 phương pháp biểu diễn văn bản:

Bảng 5: So sánh 4 phương pháp Embedding

Phương pháp	Loại	Đặc điểm	Lecture
TF-IDF	Sparse, thống kê	Baseline, không capture ngữ nghĩa	Lec02
Word2Vec	Dense, predictive	Skip-gram/CBOW, ngữ cảnh cục bộ	Lec02
GloVe	Dense, count-based	Co-occurrence matrix, ngữ nghĩa toàn cục	Lec02
PhoBERT	Dense, contextual	Transformer, tiếng Việt, SOTA	Lec06



Hình 4: So sánh Retrieval ($MRR \pm 95\%$ CI bootstrap): baseline Keyword + 4 phương pháp học, trên 100 truy vấn giàu từ khoá (trái) vs 16 truy vấn lexical-gap (phải). CI đã siết còn ± 0.05 nhưng vẫn chổng lẩn ở truy vấn dễ; chỉ ở lexical-gap PhoBERT mới tách rõ.

Biểu diễn PhoBERT (dùng cho truy xuất). Câu được tách từ rồi đưa qua PhoBERT; vector tài liệu là **trung bình có mặt nạ** của các trạng thái ẩn lớp cuối, sau

đó L_2 -chuẩn hoá để cosine quy về tích vô hướng:

$$\mathbf{e}(d) = \frac{\sum_t m_t \mathbf{h}_t}{\sum_t m_t}, \quad \hat{\mathbf{e}}(d) = \frac{\mathbf{e}(d)}{\|\mathbf{e}(d)\|_2},$$

với $\mathbf{h}_t$ là embedding token t , $m_t \in \{0, 1\}$ là mặt nạ chú ý (`max_seq_len` = 128). Token-embedding cho MaxSim (Mục 4.10) lấy tương tự nhưng giữ *từng* token (`max_len` = 64). Cấu hình huấn luyện TF-IDF / Word2Vec (SGNS) / GloVe-SVD (PPMI) được trình bày định lượng ở Thí nghiệm 5.5.

Lưu ý về độ đo: không dùng “trung bình cosine similarity” để so sánh, vì độ lớn cosine **không so sánh được** giữa các không gian embedding khác nhau (TF-IDF thừa cho giá trị thấp hơn PhoBERT dày). Thay vào đó dùng **Precision@k** và **MRR** với ground-truth là nhãn aspect (xem Mục 5.5).

Lưu ý về “GloVe”: do hạn chế tài nguyên, chúng tôi dùng **xấp xỉ kiểu GloVe** — SVD trên ma trận đồng xuất hiện ở mức tài liệu (gần với LSA), không phải GloVe của sở ngữ cảnh gốc.

4.6 Module 2b: Subword / BPE — Lec03

Các phương pháp ở trên dùng **từ vựng cố định** nên gặp từ hiếm/OOV sẽ mất biểu diễn. PhoBERT dùng **Byte-Pair Encoding (BPE)** (từ vựng 64.000 subword) tách từ thành mảnh con (hậu tố `@@` = nối tiếp). Notebook Part 1 minh hoạ trực tiếp:

Bảng 6: BPE tokenization (PhoBERT) — từ thường, từ sai chính tả, và OOV

Dầu vào	Subword BPE
camera chụp đêm	[camera, chụp, đêm]
pinnn tuttt nhanh	[pin@@, nn, tụ@@, t@@, tt, nhanh@@, h@@, h]
chống_nước	[chống_@@, nước]
xuyzzphone (OOV bịa)	[xuy@@, zz@@, phone]

Kể cả từ viết sai (“pinnn”) hay từ chưa từng thấy (“xuyzzphone”) vẫn được biểu diễn qua tổ hợp subword thay vì trở thành `<unk>` — đây là ưu điểm then chốt của subword so với TF-IDF/Word2Vec từ vựng cố định.

4.7 Module 3: Trích xuất thực thể (NER) + Gazetteer thương hiệu

Phát biểu vấn đề. NER tổng quát *underthesea* chỉ có tập nhãn {PER, LOC, ORG}; trên review TMDT nó gán chủ yếu PER/LOC và gần như **không** sinh ORG cho thương hiệu điện thoại (TN2: ORG chỉ 6/800 span). Trong khi đó node cần cho Knowledge Graph lại là **brand** — một *tập thực thể đóng, biết trước*. Công cụ phù hợp cho tập đóng là **gazetteer** (từ điển thực thể), không phải NER học máy tổng quát.

Hình thức hoá. Gazetteer là ánh xạ $\Phi: \mathcal{S} \rightarrow \mathcal{B}$ từ **16 dạng bề mặt** $\mathcal{S}$ (gồm alias và tên dòng máy) về **12 thương hiệu chuẩn** $\mathcal{B}$:

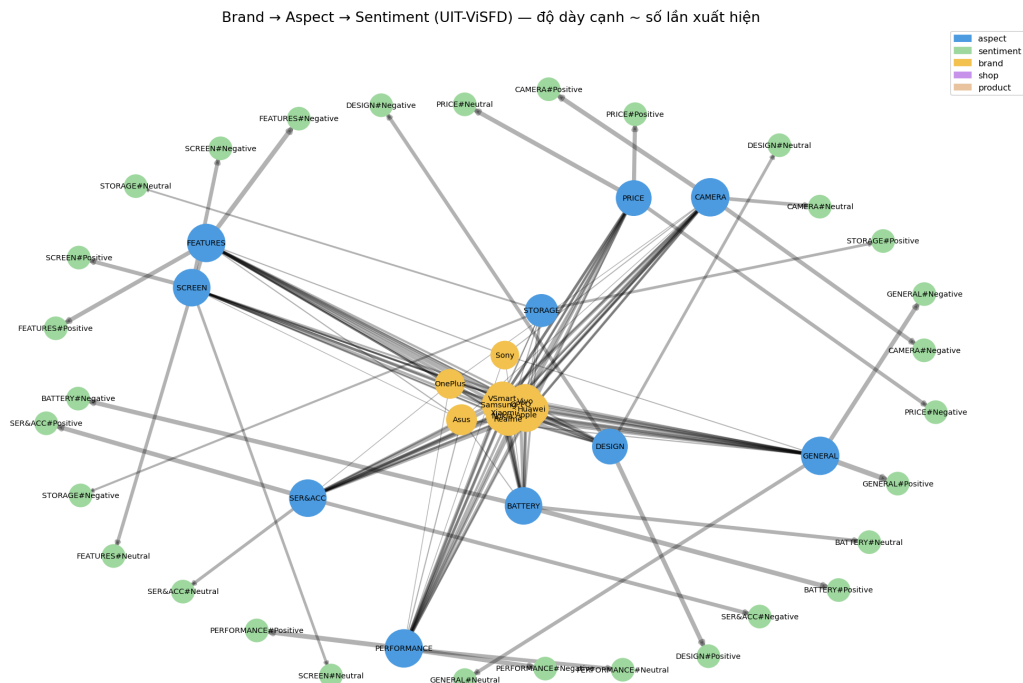
Bảng 7: Ánh xạ gazetteer Φ : 16 dạng bề mặt $\rightarrow$ 12 thương hiệu chuẩn (giải thích chênh lệch “16 key vs 12 node brand” của KG).

Thương hiệu chuẩn	Dạng bề mặt (alias)
Apple	iphone, apple
Samsung	samsung, galaxy
Xiaomi	xiaomi, redmi, poco
OPPO · Vivo · Realme	oppo · vivo · realme
Huawei · Nokia · VSmart	huawei · nokia · vsmart
Asus · Sony · OnePlus	asus · sony · oneplus

Hàm khớp $\text{detect_brand}(d)$: hạ chữ thường d , quét tuyến tính $\mathcal{S}$ và trả $\Phi(s)$ cho dạng bề mặt s đầu tiên là chuỗi con của d ; trả **Unknown** nếu không khớp. Độ phức tạp $O(|\mathcal{S}|)$ mỗi review.

Căn cứ & giới hạn. Gazetteer bắt brand ở **72/800** review (TN2), vượt xa nhãn ORG của NER tổng quát. Hạn chế (khai báo trung thực): (i) khớp chuỗi con có thể *quá khớp*; (ii) quy tắc *first-match* không phân giải khi một review nhắc nhiều hãng; (iii) phụ thuộc danh sách thủ công, không tự mở rộng. Đây là đánh đổi *precision đổi lấy đơn giản*, hợp lý ở quy mô 12 hãng đóng. (PhoNER_COVID19 không dùng vì khác domain — xem Mục 3.1.)

4.8 Module 4: Knowledge Graph



Hình 5: Knowledge Graph (102 node, 614 cạnh): $\text{brand} \rightarrow \text{aspect} \rightarrow \text{aspect\#sentiment}$, độ dày cạnh = số review. Cạnh $\text{brand} \rightarrow \text{aspect\#sentiment}$ cho phép truy vấn cảm xúc theo từng hãng cụ thể.

KG ở đây **không chỉ là hình minh hoạ** mà là một **cơ sở tri thức có cấu trúc** được truy vấn lúc suy luận. Phần này trình bày đầy đủ: (a) cách biểu diễn tri thức, (b) thuật toán xây dựng, (c) tham số & ngưỡng, (d) cách tích hợp vào RAG.

(a) Biểu diễn tri thức — lược đồ (schema)

Tri thức được biểu diễn bằng **đồ thị có hướng, có trọng số** $G = (V, E)$ (NetworkX DiGraph); mỗi cạnh là một **bộ ba** (h, r, t) kèm **trọng số** $w = \text{số review chứng thực}$ quan hệ đó. Vd. bộ ba thực (Samsung, brand_sentiment, BATTERY#Positive, $w=68$) đọc là “68 review của Samsung khen pin”.

Bảng 8: Lược đồ KG: 5 loại node và quan hệ (cạnh). Trọng số mọi cạnh = số review.

Loại ($ V $)	node	Thuộc tính	Ý nghĩa
brand (12)	—	—	thương hiệu (từ gazetteer)
aspect (10)	—	—	khía cạnh, gồm SER&ACC
aspect#sentiment (30)	—	—	cặp (khía cạnh, cảm xúc) = 10×3
shop (24)	—	—	gian hàng Shopee
product (26)	—	avg_rating, n_reviews	sản phẩm Shopee
Quan hệ ($h \rightarrow t$)	r	Ngữ nghĩa (trọng số = số review)	
reviewed_on	brand→aspect	—	hãng được nhắc về khía cạnh
has_sentiment	aspect→aspect#sentiment	—	phân bố cảm xúc TOÀN corpus (fallback)
brand_sentiment	brand→aspect#sentiment	—	cảm xúc RIÊNG từng hãng (cạnh then chốt)
sells	shop→product	—	—
mentions	product→aspect	—	aspect do BiLSTM dự đoán

Thiết kế then chốt: cạnh **brand_sentiment** cho phép truy vấn cảm xúc **theo từng hãng**: hỏi “pin Samsung” trả thống kê riêng Samsung ($n=126$) thay vì gộp toàn corpus ($n=3.604$) — câu trả lời đúng trọng tâm thay vì chung chung.

(b) Thuật toán xây dựng (offline)

Algorithm 1 BuildKG — dựng Knowledge Graph từ review (hiện thực `core/kg.py`)

```

1:  $G \leftarrow$  DiGraph rỗng; thêm 10 node aspect
2: for mỗi review  $d$  trong UIT-ViSFD (nhãn vàng) do
3:    $b \leftarrow$  DETECTBRAND( $d$ ) ▷ gazetteer 16 brand
4:   for mỗi cặp  $(\alpha, s) \in$  PARSELABEL( $d$ ) do
5:     bump cạnh  $\alpha \rightarrow \alpha \# s$  (has_sentiment, toàn corpus)
6:     if  $b \neq$  Unknown then
7:       bump  $b \rightarrow \alpha$  (reviewed_on); bump  $b \rightarrow \alpha \# s$  (brand_sentiment)
8:     end if
9:   end for
10: end for
11: for mỗi review  $d$  trong Shopee (KHÔNG nhãn) do
12:    $A \leftarrow$  BiLSTM.PREDICT( $d$ ) ▷ aspect dự đoán, ngưỡng sigmoid 0.5
13:   thêm node shop, product; bump shop $\rightarrow$ product (sells)
14:   for mỗi  $\alpha \in A$  do bump product $\rightarrow \alpha$  (mentions)
15:   end for
16:   cập nhật product.avg_rating, n_reviews
17: end for
18: return  $G$  ▷ lưu kg.pkl, gán version theo (model, #record)

```

(c) Tham số & ngưỡng — “build tới ngưỡng nào thì tối ưu”

Bảng 9: Các ngưỡng/tham số khi xây & dùng KG, kèm căn cứ.

Tham số	Giá trị	Vai trò & căn cứ
τ_{conf} (MIN_OK)	30	n tối thiểu để một thống kê cảm xúc theo hãng <i>đáng tin</i> ; dưới ngưỡng $\Rightarrow$ gán cờ “mẫu nhỏ” (xem căn cứ thống kê bên dưới).
Ngưỡng BiLSTM	0.5	ngưỡng sigmoid gán aspect đa nhãn cho review Shopee (<code>aspect_clf.py</code>).
Trùng token sản phẩm	≥ 2	chống khớp giả product $\leftrightarrow$ câu hỏi do 1 token chung; có regression test (<code>test_kg.py</code>).
Gazetteer brand	16 key	nguồn tạo node brand vì NER tổng quát gần như không sinh ORG (TN2).

Vì sao $\tau_{\text{conf}} = 30$? Với ước lượng tỉ lệ $\hat{p}$ (vd. % tích cực), nửa độ rộng khoảng tin cậy 95% xấp xỉ $1.96\sqrt{\hat{p}(1-\hat{p})/n}$. Tại $n = 30$ (xấu nhất $\hat{p}=0.5$) sai số $\approx \pm 0.18$; tại $n = 126$ (pin Samsung) còn ± 0.087 . Vậy 30 là **sàn hợp lý**: dưới mức này % cảm xúc dao động quá lớn nên hệ thống gán cờ “mẫu nhỏ” thay vì khẳng định (hiện thực `confidence_note`, dùng ở `pipeline._graph_context`). Mức “tối ưu” của KG vì thế **bị chặn bởi dữ liệu**, không phải tham số: chỉ 47/120 ô hãng $\times$ aspect đạt τ_{conf} (xem Mục 4.9).

(d) Tích hợp vào RAG (lúc truy vấn)

Hàm `_graph_context` (`rag/pipeline.py`) biến KG thành ngữ cảnh cho LLM:

1. $\alpha \leftarrow \text{aspect_from_query}(q)$; $b \leftarrow \text{detect_brand}(q)$.
2. Nếu b xác định: $sd \leftarrow \text{graph_query_brand}(b, \alpha)$; nếu rỗng $\rightarrow$ fallback $\text{graph_query}(\alpha)$ toàn corpus.
3. Tính % cảm xúc từ trọng số cạnh; nếu $n < \tau_{\text{conf}}$ hoặc $\alpha = \text{STORAGE} \Rightarrow$ thêm cảnh báo độ tin cậy.
4. $\text{product_context}(q)$ cho câu hỏi đa sản phẩm (yêu cầu ≥ 2 token trùng).
5. Ghép thông kê vào **prompt grounded** $\rightarrow$ LLM (Module 6).

4.9 Độ đủ của dữ liệu cho Knowledge Graph

Một câu hỏi phản biện: *dữ liệu có đủ dày để KG đạt ngưỡng tin cậy $\tau_{\text{conf}}=30$ (Mục (c)) không?* Ta phân tích trực tiếp KG (`scripts/analyze_kg.py`): 102 node (12 brand, 10 aspect, 30 aspect#sentiment, 24 shop, 26 product), 614 cạnh.

Bảng 10: Độ phủ dữ liệu KG. “ ≥ 30 ” = đủ review để một thống kê cảm xúc theo hãng đáng tin.

Chỉ số độ đủ	Giá trị
Hãng có ≥ 100 lượt nhắc	9 / 12 (Samsung 1029, Xiaomi 599, OPPO 598, Apple 593. ...)
Hãng quá thưa (< 30 ở mọi aspect)	Nokia, Huawei, Asus, Sony, OnePlus
Ô (hãng $\times$ aspect) đủ ≥ 30 review	47 / 120 (có ≥ 1 review: 105/120)
Aspect dày nhất / mỏng nhất	GENERAL 4.866 ... STORAGE chỉ 9
Sản phẩm Shopee có ≥ 20 review	20 / 2

Kết luận trung thực: KG đủ dày cho hãng phổ biến (Samsung/Xiaomi/OPPO/Apple — 7–9/10 aspect đạt ≥ 30) và các aspect thường (BATTERY 3.604, PERFORMANCE 4.140...). Nhưng **thưa ở hai chỗ**: (i) *hãng nhỏ* (Nokia/Huawei/Asus/Sony/OnePlus dưới ngưỡng ở mọi aspect $\Rightarrow$ truy vấn theo hãng buộc fallback về thống kê toàn corpus); (ii) *aspect STORAGE* chỉ 91 lượt toàn corpus — và đúng là aspect mà BiLSTM cũng thất bại ($F1=0$, Mục 5.8). Chỉ **47/120 ô hãng $\times$ aspect** đủ tin cậy. Đây là giới hạn dữ liệu cần khắc phục khi mở rộng (Hướng phát triển): thu thêm review hãng nhỏ và đặc biệt nhãn STORAGE.

4.10 Module 5: Truy xuất lai hai tầng với tái xếp hạng late-interaction

Đây là thành phần lõi của “Graph RAG”. Truy xuất gồm **hai tầng**: sinh ứng viên (dense $\cup$ sparse) để tối đa *recall*, rồi tái xếp hạng bằng tổ hợp bốn tín hiệu để tối đa *precision*.

(a) Tầng 1 — Sinh ứng viên

Cho truy vấn q và corpus $\mathcal{D}$:

- *Dense* (PhoBERT bi-encoder): $\mathcal{C}_{\text{dense}} = \text{top-}N \{ \cos(\hat{\mathbf{e}}(q), \hat{\mathbf{e}}(d)) \}$ với $\hat{\mathbf{e}}$ là vector mean-pool đã L_2 -chuẩn hoá (Mục 4.5).

- *Sparse* (BM25-Okapi [10]): $\mathcal{C}_{\text{bm25}} = \text{top-}N\{\text{BM25}(q, d)\}$ — bắt **khớp chính xác** (mã model, “120hz”) mà dense dễ bỏ sót.
- **Hợp**: $\mathcal{C} = \mathcal{C}_{\text{dense}} \cup \mathcal{C}_{\text{bm25}}$ (tối đa $2N$ ứng viên), $N = 50$.

(b) Tầng 2 — Tái xếp hạng bốn tín hiệu

Với mỗi $d \in \mathcal{C}$ tính bốn điểm thành phần:

$$\begin{aligned} s_{\text{bi}}(d) &= \cos(\hat{\mathbf{e}}(q), \hat{\mathbf{e}}(d)), & s_{\text{bm25}}(d) &= \text{BM25}(q, d), \\ s_{\text{attn}}(d) &= \text{MaxSim}(q, d) = \frac{1}{|q|} \sum_{i \in q} \max_{j \in d} \mathbf{t}_i^q \cdot \mathbf{t}_j^d, & s_{\text{graph}}(d) &= \mathbf{1}[\alpha(q) \in \mathcal{A}(d)]. \end{aligned}$$

Trong đó $\mathbf{t}^q, \mathbf{t}^d$ là embedding token đã L_2 -chuẩn hoá (tích vô hướng = cosine); MaxSim là **late-interaction kiểu ColBERT** [9], *không tham số và không cần huấn luyện*. $\alpha(q)$ là aspect suy ra từ truy vấn (khớp từ khoá), $\mathcal{A}(d)$ là tập aspect *quan sát trong nội dung* d — **độc lập với nhãn vàng dùng để chấm**, nên graph boost **không rò rỉ metric** (đây là điểm mấu chốt cho tính hợp lệ của TN3).

(c) Chuẩn hoá & kết hợp

Mỗi thành phần liên tục được min-max chuẩn hoá về $[0, 1]$ trên tập $\mathcal{C}$, $\tilde{s} = (s - \min)/(\max - \min)$, để các thang đo khác nhau *so sánh được* trước khi cộng tuyến tính:

$$\text{score}(d) = w_{\text{bi}} \tilde{s}_{\text{bi}}(d) + w_{\text{attn}} \tilde{s}_{\text{attn}}(d) + w_{\text{graph}} s_{\text{graph}}(d) + w_{\text{bm25}} \tilde{s}_{\text{bm25}}(d),$$

trả về top- k theo score.

Algorithm 2 HybridRetrieve — truy xuất lai 2 tầng (rag/retrieval.py)

Require: truy vấn q , corpus $\mathcal{D}$, trọng số $(w_{\text{bi}}, w_{\text{attn}}, w_{\text{graph}}, w_{\text{bm25}})$

- 1: $\mathcal{C}_{\text{dense}} \leftarrow \text{top-}N$ theo $\cos(\hat{\mathbf{e}}(q), \hat{\mathbf{e}}(d))$; $\mathcal{C}_{\text{bm25}} \leftarrow \text{top-}N$ theo $\text{BM25}(q, d)$
 - 2: $\mathcal{C} \leftarrow \mathcal{C}_{\text{dense}} \cup \mathcal{C}_{\text{bm25}}$
 - 3: **for** $d \in \mathcal{C}$ **do**
 - 4: $s_{\text{bi}}, s_{\text{bm25}} \leftarrow \text{cosine}, \text{BM25}$ ▷ tính cho toàn $\mathcal{C}$
 - 5: $s_{\text{attn}} \leftarrow \text{MAXSIM}(q, d)$ nếu $d \in \mathcal{C}_{\text{dense}}$, ngược lại 0 ▷ token-encode tốn kém
 - 6: $s_{\text{graph}} \leftarrow \mathbf{1}[\alpha(q) \in \mathcal{A}(d)]$
 - 7: **end for**
 - 8: min-max chuẩn hoá $s_{\text{bi}}, s_{\text{attn}}, s_{\text{bm25}}$ trên $\mathcal{C}$
 - 9: **return** top- k theo $w_{\text{bi}}\tilde{s}_{\text{bi}} + w_{\text{attn}}\tilde{s}_{\text{attn}} + w_{\text{graph}}s_{\text{graph}} + w_{\text{bm25}}\tilde{s}_{\text{bm25}}$
-

Bảng 11: Tham số truy xuất (config.yaml).

Tham số	Giá trị	Vai trò
N (n_candidates)	50	số ứng viên mỗi nhánh dense/sparse
k (top_k)	5	số review đưa sang Generation
w_{bi}	0.35	trọng số cosine bi-encoder
w_{attn}	0.35	trọng số MaxSim (late-interaction)
w_{graph}	0.15	trọng số graph boost (nhị phân)
w_{bm25}	0.15	trọng số BM25

Tối ưu chi phí: MaxSim phải mã hoá token nên (i) *chỉ* tính cho $\mathcal{C}_{\text{dense}}$ (gán 0 cho ứng viên chỉ-BM25), và (ii) memoize embedding token theo `doc_idx` — các ứng viên phổ biến lặp lại nhiều truy vấn chỉ encode một lần. Trọng số trên là giá trị tune-trên-dev (Mục 6.2), không tune trên tập báo cáo.

4.11 Module 6: Sinh câu trả lời có kiểm soát (grounded generation)

Mục tiêu: sinh câu trả lời tiếng Việt **neo chặt vào bằng chứng** (top- k review + thống kê Knowledge Graph) nhằm *giảm bịa đặt*.

Dựng ngữ cảnh. Cho tập review $\mathcal{R}$ (top- k , Mục 4.10) và chuỗi thống kê KG g (Mục 4.8), prompt được ghép từ ba khối: (i) mỗi review cắt còn 240 ký tự, gắn tiền tố nguồn [UIT-ViSFD|Shopee]; (ii) khối thống kê KG; (iii) câu hỏi.

Chính sách kiểm soát hai lớp. (1) *Grounding* — chỉ thị “CHỈ dựa trên dữ liệu dưới đây”. (2) *Abstention* — nếu dữ liệu không chứa thông số kỹ thuật được hỏi (IP68, Hz, GHz, mAh, gram, số năm bảo hành, phút sạc, độ phân giải) thì **bắt buộc** trả “Đánh giá không đề cập thông tin này”, tuyệt đối không suy ra từ tham số mô hình.

Listing 3: Prompt grounded (rút gọn từ `rag/generate.py`).

```

1 [SYSTEM] Ban la tro ly tu van mua sam online, tra loi bang tieng Viet.
2
3 [USER]
4 CHI dua tren du lieu duoi day, tra loi ngan gon bang tieng Viet.
5 QUAN TRONG: neu du lieu KHONG chua thong so ky thuat duoc hoi
6 (vd. IP68, Hz, GHz, mAh, gram, so nam bao hanh, phut sac, do phan giai)
7 PHAI tra loi 'Danh gia khong de cap thong tin nay' -- khong bia.
8
9 === DANH GIA NGUOI DUNG ===
10 [UIT-ViSFD] <review 1 ... (<=240 ky tu)>
11 [Shopee] <review 2 ...>
12 === THONG KE TU KNOWLEDGE GRAPH ===
13 <vd. BATTERY (Samsung): 54% tích cuc / 37% tieu cuc (n=126)>
14 === CAU HOI ===
15 <q>
16 Tra loi:

```

Giải mã & vận hành. Model `gpt-4o-mini`, temperature 0.3 (giảm ngẫu nhiên), `max_tokens= 400`. **Fallback:** khi thiếu `OPENAI_API_KEY`, hệ thống trả thẳng các review truy xuất thay vì báo lỗi (vẫn hữu dụng mà không cần LLM). Mỗi lần gọi được ghi log latency + token + chi phí USD (Mục 7).

5. Thực nghiệm

5.1 Môi trường thực nghiệm

Bảng 12: Cấu hình môi trường thực nghiệm

Thành phần	Chi tiết
Platform	Kaggle Notebook (T4 GPU)
Python	3.10+
PyTorch	2.x
Transformers	4.40+
LLM	OpenAI API (gpt-4o-mini)
Các thư viện khác	underthesea, gensim, networkx, openai, gradio

5.2 Giao thức đánh giá

Mọi thí nghiệm tuân theo một giao thức cố định để bảo đảm **công bằng** và **tái lập**.

Kho truy xuất (pool). Mọi thí nghiệm xếp hạng trên **cùng pool 10.923 review** = 7.786 UIT-ViSFD (nhân aspect vàng) $\cup$ 3.137 review Shopee (đóng vai *nhiều*, không liên quan aspect smartphone).

Phán quyết liên quan. Review d *liên quan* với truy vấn nhắm aspect α khi và chỉ khi $\alpha \in \text{gold}(d)$ (nhân vàng UIT-ViSFD). **Cùng một phán quyết áp cho mọi phương pháp** $\Rightarrow$ so sánh công bằng.

Không rò rỉ metric. Trong ablation, graph boost dùng aspect *quan sát trong nội dung* ($\mathcal{A}(d)$, khớp từ khoá), còn phán quyết liên quan dùng *nhân vàng* ($\text{gold}(d)$) — hai nguồn độc lập (Mục 4.10).

Bảng 13: Các bộ truy vấn đánh giá (cùng pool 10.923 review).

Bộ truy vấn	Kích thước	Mục đích	Dùng
Keyword-rich	100 (10/as- pect)	so embedding khi truy vấn giàu từ khóá	TN1, TN3
Lexical-gap	16	0 rò rỉ từ khoá (kiểm bằng code) — đo vượt lexical gap	TN1b
Grounded + ngoài phạm vi	30 + 8	faithfulness, numeric-grounding, ab- stention	TN5
Dev / Test	50 / 50	phân tầng aspect — tune trọng số không rò rỉ	Mục 6.2

Khoảng tin cậy. CI 95% của MRR ước lượng bằng **bootstrap**: resample tập truy vấn (có hoàn lại) $B = 2000$ lần, lấy phân vị 2,5%–97,5%.

Tách dev/test. Trọng số hybrid tune trên **50 dev**, báo cáo trên **50 test** (phân tầng aspect); mức thổi phồng do tune-trên-test đo riêng (Mục 6.2).

5.3 Độ đo (định nghĩa hình thức)

Đặt Q là tập truy vấn, $\text{rel}_q(d) \in \{0, 1\}$ là phán quyết liên quan, rank_q là vị trí review liên quan *đầu tiên*.

Truy xuất:

$$\begin{aligned} \text{P@}k &= \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{k} \sum_{i=1}^k \text{rel}_q(d_i), & \text{MRR} &= \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \\ \text{DCG@}k &= \sum_{i=1}^k \frac{\text{rel}_q(d_i)}{\log_2(i+1)}, & \text{NDCG@}k &= \frac{\text{DCG@}k}{\text{IDCG@}k} \in [0, 1]. \end{aligned}$$

Phân loại aspect (đa nhãn), với tập aspect A ($|A| = 10$):

$$\text{F1}_{\text{micro}} = \frac{2 \sum_a TP_a}{2 \sum_a TP_a + \sum_a FP_a + \sum_a FN_a}, \quad \text{F1}_{\text{macro}} = \frac{1}{|A|} \sum_{a \in A} \text{F1}_a.$$

Micro gộp mọi nhãn (bị aspect phổ biến thống trị); macro trung bình đều theo aspect (phơi bày lớp hiếm) — lý do báo cáo *cả hai* (Mục 5.8).

Chất lượng câu trả lời (TN5): *faithfulness* $\in [0, 100]$ (LLM-judge: % nội dung câu trả lời được bằng chứng review+KG chứng minh); *relevance* $\in \{1, \dots, 5\}$;

$$\text{numeric-grounding} = \frac{\#\{\text{số trong câu trả lời có trong bằng chứng}\}}{\#\{\text{số trong câu trả lời}\}}, \quad \text{abstention} = \frac{\#\{\text{câu ngoài phạm vi}\}}{\#\{\text{câu ngoài phạm vi}\}}$$

5.4 Quy trình tái lập

Pipeline tách **offline** (dựng artifact có version) khỏi **online** (nạp lại để serve/eval). Mọi bảng/hình trong báo cáo sinh từ artifact .json; bảng dưới ánh xạ *lệnh* $\rightarrow$ *artifact* $\rightarrow$ *nơi dùng* (lệnh module có tiền tố python -m vngraphrag.; script chạy python <path>).

Bảng 14: Tái lập: lệnh $\rightarrow$ artifact $\rightarrow$ nơi dùng trong báo cáo.

Lệnh	Artifact	Dùng ở
cli.build_index	doc_vectors.npy, kg.pkl, manifest.json	toàn hệ
cli.train_aspect	aspect_clf.pt	TN2b
cli.evaluate (+ gate)	metrics.json	TN3, TN2b
scripts/eval_embeddings_full.py	embeddings_full.json	TN1, TN1b
scripts/eval_baseline_random.py	baseline_random.json	sàn ngẫu nhiên
scripts/eval_devtest_split.py	devtest_eval.json	overfit (Mục 6.2)
scripts/eval_rag_modes.py [†]	rag_modes_eval.json	TN5
scripts/eval_grounding.py [†]	grounding_eval.json	Mục 6.2
scripts/eval_ner.py	ner_stats.json	TN2
scripts/analyze_kg.py	kg_stats.json	Mục 4.9
scripts/make_report_figs.py	figures/*.png	các hình

[†] cần OPENAI_API_KEY. Chạy toàn bộ: `make eval-all`.

Cổng hồi quy (regression gate): cli.evaluate báo lỗi nếu $\text{MRR} < 0,55$ hoặc $\text{micro-F1} < 0,70$ (ngưỡng ở config.yaml) — chặn việc commit làm tụt chất lượng.

5.5 Thí nghiệm 1: So sánh Embedding Methods

Thiết lập: 100 truy vấn phủ đủ 10 aspect (10 câu/aspect; mở rộng từ 8→25→100 để siết khoảng tin cậy, dùng cùng bộ truy vấn với Thí nghiệm 3). Pool xếp hạng là toàn corpus 10.923 review (7.786 UIT-ViSFD có nhãn vàng + 3.137 review Shopee đóng vai nhiễu, không liên quan). Một review được coi là *liên quan* nếu nhãn vàng của nó chứa aspect đó. Báo cáo Precision@5, Precision@10, MRR, kèm **khoảng tin cậy 95% bootstrap** (2.000 lần resample truy vấn) để biết chênh lệch có vượt nhiều không. Cùng một phán quyết liên quan áp dụng cho mọi phương pháp $\Rightarrow$ công bằng. Ta bổ sung **baseline “Keyword”** — xếp hạng theo *số từ trùng lặp thô* giữa truy vấn và review (mô phỏng ô tìm kiếm từ khoá của sàn TMDT, **không** ngữ nghĩa, **không** IDF) — để đối chiếu trực tiếp “không dùng học biểu diễn” với “dùng”. Word2Vec huấn luyện skip-gram + negative sampling (PyTorch); GloVe-SVD lấy từ SVD ma trận PPMI đồng xuất hiện; PhoBERT dùng vector mean-pool.

Bảng 15: Precision@k & MRR theo phương pháp (pool 10.923 review, **100 truy vấn giàu từ khoá**). Cột cuối: khoảng tin cậy 95% bootstrap của MRR. Baseline ngẫu nhiên là *sàn tham chiếu*.

Phương pháp	P@5	P@10	MRR	MRR 95% CI
<i>Ngẫu nhiên (sàn)</i>	0.219	0.220	0.416	—
<i>Keyword (đối chiếu)</i>	0.798	0.768	0.866	[0.810, 0.916]
TF-IDF	0.722	0.704	0.817	[0.758, 0.878]
Word2Vec	0.748	0.743	0.819	[0.757, 0.880]
GloVe-SVD	0.766	0.766	0.826	[0.759, 0.886]
PhoBERT	0.678	0.667	0.821	[0.763, 0.880]

Mọi phương pháp đều **vượt xa sàn ngẫu nhiên** (MRR 0.416), khẳng định tín hiệu truy xuất là thực. Với **100 truy vấn**, các khoảng tin cậy đã **siết lại còn ± 0.05** (so với ± 0.15 ở 25 câu) — nhưng chúng **vẫn chồng lấn nhau gần như hoàn toàn** (mọi MRR nằm trong 0.76–0.89). Kết luận trung thực: trên truy vấn *giàu từ khoá*, **không phương pháp học nào tách biệt có ý nghĩa thống kê**, và baseline **Keyword thuần thậm chí có MRR/P@k cao nhất** — vì khi câu hỏi chứa thẳng từ khoá aspect thì *trùng lặp từ vựng đã đủ*, không cần ngữ nghĩa; PhoBERT mean-pool (*không fine-tune*) còn kém ở P@5 (0.678). Tức “dùng học biểu diễn” **chưa thắng “dùng từ khoá” khi truy vấn dễ**. Câu chuyện đối chiều hoàn toàn ở Thí nghiệm 1b — đúng nỗi đau *lexical gap* của bài toán.

5.6 Thí nghiệm 1b: Truy vấn có *lexical gap* thật

Để kiểm tra đúng khả năng vượt **lexical gap**, chúng tôi xây bộ **16 truy vấn đời thường KHÔNG chứa từ khoá của aspect đích** (vd. hỏi về pin nhưng không có chữ “pin/sạc”: “*dùng cả ngày trời mà vẫn chưa cần cắm điện*”). Mỗi câu được code kiểm tra **0 rò rỉ từ khoá**; pool xếp hạng và phán quyết liên quan giữ **nguyên như Thí nghiệm 1** nên so sánh được trực tiếp (tái lập bằng `scripts/eval_lexical_gap.py`).

Bảng 16: Retrieval trên 16 truy vấn *lexical gap* (pool 10.923 review). Cột cuối: khoảng tin cậy 95% bootstrap của MRR.

Phương pháp	P@5	P@10	MRR	MRR 95% CI
Keyword (đối chiếu)	0.537	0.562	0.736	[0.561, 0.906]
TF-IDF	0.500	0.438	0.572	[0.394, 0.753]
Word2Vec	0.512	0.481	0.656	[0.453, 0.859]
GloVe-SVD	0.525	0.512	0.629	[0.433, 0.803]
PhoBERT	0.637	0.619	0.776	[0.595, 0.938]

Kết quả quyết định — nhìn ở độ chính xác *top-k*: khi không còn từ khoá trùng, **P@5/P@10** của TF-IDF thuần từ vựng **sụp đổ** (0.500/0.438), Keyword cũng chỉ còn 0.538/0.563; trong khi **PhoBERT vượt rõ mọi phương pháp** (0.638/0.619). Đây là bằng chứng cho ưu thế *ngữ nghĩa*: trên đúng nỗi đau *lexical gap*, biểu diễn theo nghĩa thẳng biểu diễn theo từ vựng.

Một cảnh báo trung thực về MRR: baseline Keyword đạt **MRR 0.736** (cao thứ 2) dù P@5 chỉ 0.538. Lý do: MRR chỉ thưởng *một* review đúng nằm gần đầu, mà trên pool 10.923 với aspect phổ biến, chỉ cần trùng vài từ phụ (“cả ngày”, “ổ điện”) là một review liên quan lọt top. Vì vậy **MRR đơn lẻ phóng đại** các baseline yếu — phải đọc **cùng P@k**. Cảnh báo này chính là một phần câu trả lời cho “metrics có đúng không” (Mục 6.2). Lưu ý thêm: mọi CI vẫn còn rộng (16 truy vấn) nên các kết luận cần được xác nhận lại trên bộ lớn hơn.

5.7 Thí nghiệm 2: Trích xuất thực thể (NER)

Thiết lập: chạy NER tổng quát (*underthesea*) trên **800 review mẫu** trải đều corpus, đếm số *span* thực thể theo loại; song song nhận diện thương hiệu bằng **gazetteer**. Vì domain TMDT không có nhãn vàng NER nên báo cáo theo **số lượng & độ phủ** (định tính).

Bảng 17: Thống kê NER trên 800 review mẫu (số span thực thể & độ phủ brand)

Loại thực thể / nguồn	Số lượng
PER (<i>underthesea</i>)	292 span
LOC (<i>underthesea</i>)	282 span
ORG (<i>underthesea</i>)	6 span
Review có ≥ 1 thực thể	387 / 800
Review nhận diện được brand (<i>gazetteer</i>)	72 / 800
Top brand	Samsung 23 · Apple 16 · Xiaomi 7 · VSmart 7

Quan sát: NER tổng quát chủ yếu gán **PER/LOC** (gần 575 span), trong khi **ORG rất hiếm (6 span)** — mô hình tổng quát *không* nhận các thương hiệu điện thoại là tổ chức. Đây chính là lý do thiết kế cần **gazetteer thương hiệu**: nó bắt được brand ở 72/800 review và là nguồn tạo node **brand** cho Knowledge Graph, thay vì dựa vào nhãn ORG của NER tổng quát.

5.8 Thí nghiệm 2b: Phân loại aspect bằng RNN (BiLSTM)

Hiện thực **Lec05 (Recurrent Neural Networks)**: một **BiLSTM** (embedding 100, hidden 128, hai chiều) phân loại **đa nhãn** 10 aspect từ review, huấn luyện trên tập train UIT-ViSFD và đánh giá **micro-F1** trên tập dev. Mô hình sau khi train được **lưu lại** (`artifacts/aspect_clf.pt`) và **triển khai** trong hệ thống (xem Mục 7): dùng để gán aspect cho review Shopee (không có nhãn) nhằm làm giàu Knowledge Graph, và phục vụ trực tiếp qua API `POST /classify`.

Micro-F1 (dev): 0.79 · Macro-F1 (dev): 0.68 (8 epoch trên 7.786 review train, đánh giá trên 1.112 review dev; lưu `artifacts/aspect_clf.pt`).

Vì sao báo cáo CẢ macro-F1? Đây là một phần trả lời cho “metrics có đúng không” (Mục 6.2): **micro-F1 (0.79) gộp mọi nhãn nên bị thống trị bởi các aspect phổ biến** và *che giấu* aspect hiếm. Macro-F1 (0.68) — trung bình F1 đều theo aspect — phơi bày sự chênh lệch:

Bảng 18: F1 từng aspect của BiLSTM trên tập dev (sắp theo F1). Micro-F1 0.79 *che giấu* việc **STORAGE hoàn toàn thất bại**.

Aspect	F1	Recall	Aspect	F1	Recall
BATTERY	0.90	0.86	PRICE	0.79	0.81
CAMERA	0.83	0.82	FEATURES	0.76	0.78
GENERAL	0.81	0.86	SER&ACC	0.74	0.69
PERFORMANCE	0.79	0.78	DESIGN	0.64	0.53
			SCREEN	0.51	0.40
			STORAGE	0.00	0.00

Phát hiện trung thực: STORAGE có F1 = 0 — mô hình *không bao giờ* dự đoán đúng aspect này (lớp hiếm nhất, bị nuốt bởi mất cân bằng nhãn), và SCREEN recall chỉ 0.40. Một báo cáo chỉ nêu micro-F1 0.79 sẽ **giấu mất thất bại này**. Đây là lý do dùng macro-F1 và bảng per-aspect; hướng khắc phục (class weighting / fine-tune PhoBERT) ở Mục Hạn chế.

5.9 Thí nghiệm 3: Ablation chiến lược Retrieval (không rò rỉ metric)

Thiết lập: 100 truy vấn, trên cùng tập Top-50 candidate, thêm dần **4 thành phần** (bi-encoder → +MaxSim → +graph boost → +BM25). **Quan trọng:** graph boost dựa trên aspect *quan sát trong nội dung review* (tín hiệu khả dụng lúc suy luận), còn phán quyết liên quan dùng *nhãn vàng* — hai nguồn độc lập nên không có rò rỉ. Thêm **NDCG@5** (chỉ số IR chuẩn thưởng cả thứ hạng).

Bảng 19: Ablation: đóng góp tăng dần của từng thành phần (100 truy vấn, Top-50 candidate). Δ = thay đổi MRR so với dòng trên.

Cấu hình	P@5	P@10	MRR	NDCG@5	Δ MRR
<i>Ngẫu nhiên (sàn)</i>	0.219	0.220	0.416	—	—
Bi-encoder (PhoBERT)	0.678	0.667	0.821	0.688	+0.405 vs sàn
+ Attention (MaxSim)	0.704	0.698	0.812	0.704	−0.010
+ Graph boost (Graph RAG)	0.742	0.750	0.845	0.744	+0.033
+ BM25 (full hybrid)	0.748	0.763	0.838	0.748	−0.007

Nhận xét: graph boost là đóng góp lớn nhất và *robust* — nâng MRR từ 0.812 lên **0.845 (+0.033)** và cải thiện **mọi** chỉ số (P@5 0.704→0.742, NDCG@5 0.704→0.744). Đây là tín hiệu chắc chắn hơn nhiều so với +0.013 ở bộ 25 câu, khẳng định giá trị thật của thành phần “Graph” trong Graph RAG. MaxSim đánh đổi một chút MRR (−0.010) lấy độ chính xác top-k cao hơn (P@5/NDCG tăng). Thêm BM25 cho **P@10/NDCG@5 cao nhất** (0.763/0.748) nhưng hơi giảm MRR: không có cấu hình “thắng mọi chỉ số” — tùy ưu tiên 1-hit (MRR) hay phủ top-k (P@10/NDCG).

5.10 Thí nghiệm 4: Demo End-to-End

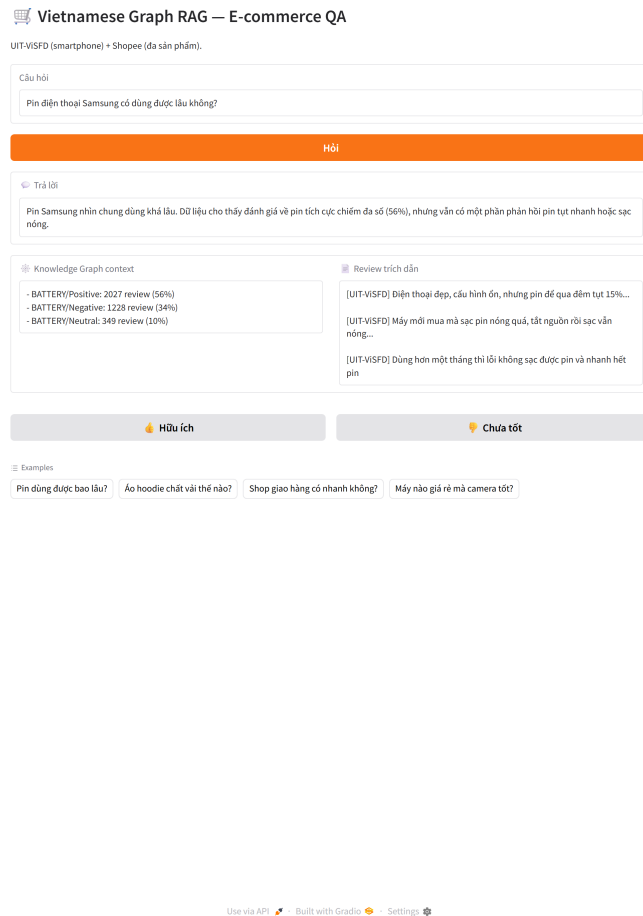
Ba câu hỏi mẫu chạy qua pipeline đầy đủ Retrieve → Graph context → sinh câu trả lời (model **gpt-4o-mini** qua OpenAI API). Mỗi câu trả lời được **grounding** vào Top-5 review truy xuất và thống kê cảm xúc từ Knowledge Graph; toàn bộ latency/token/chi phí được ghi log (Mục 7).

Bảng 20: Demo end-to-end: câu hỏi → thống kê KG → câu trả lời sinh ra (grounded)

Câu hỏi	Thống kê KG (grounding)	Câu trả lời hệ thống
Camera điện thoại chụp đêm có tốt không?	CAMERA: 57% tích cực / 29% tiêu cực (n=2.146)	“Có, theo đánh giá thì camera chụp đêm khá tốt.”
Pin Samsung dùng có lâu không?	BATTERY (riêng Samsung): 54% tích cực / 37% tiêu cực (n=126)	“Pin nhìn chung dùng khá lâu vì đa số đánh giá tích cực; tuy vậy vẫn có phản hồi pin tụt nhanh, sạc nóng.”
Dịch vụ bảo hành và nhân viên tư vấn thế nào?	SER&ACC: 70% tích cực / 24% tiêu cực (n=1.995)	“Bảo hành và hậu mãi nhìn chung khá tốt, được khen; song vẫn có ý kiến chê bảo hành kém, tư vấn bán hàng thì ổn.”

Nhận xét: câu trả lời bám sát phân bố cảm xúc trong KG (vd. SER&ACC 70% tích cực → trả lời “khá tốt” nhưng vẫn nêu mặt tiêu cực 24%), minh họa đúng mục tiêu *giảm bias đặt* bằng grounding. Riêng câu hỏi nhắc tên hãng, thống kê được **lọc theo đúng hãng** (Samsung n=126) thay vì gộp toàn corpus — nhờ cạnh **brand→aspect#sentiment**

trong KG (Mục 4.7). Độ trễ trung bình $\approx 4\text{--}7$ giây/câu (gồm thời gian gọi LLM). *Lưu ý*: cột thống kê KG lấy trực tiếp từ `kg_stats.json`; phần *câu trả lời* là một lần chạy đại diện (LLM có tính ngẫu nhiên nên có thể khác giữa các lần gọi).

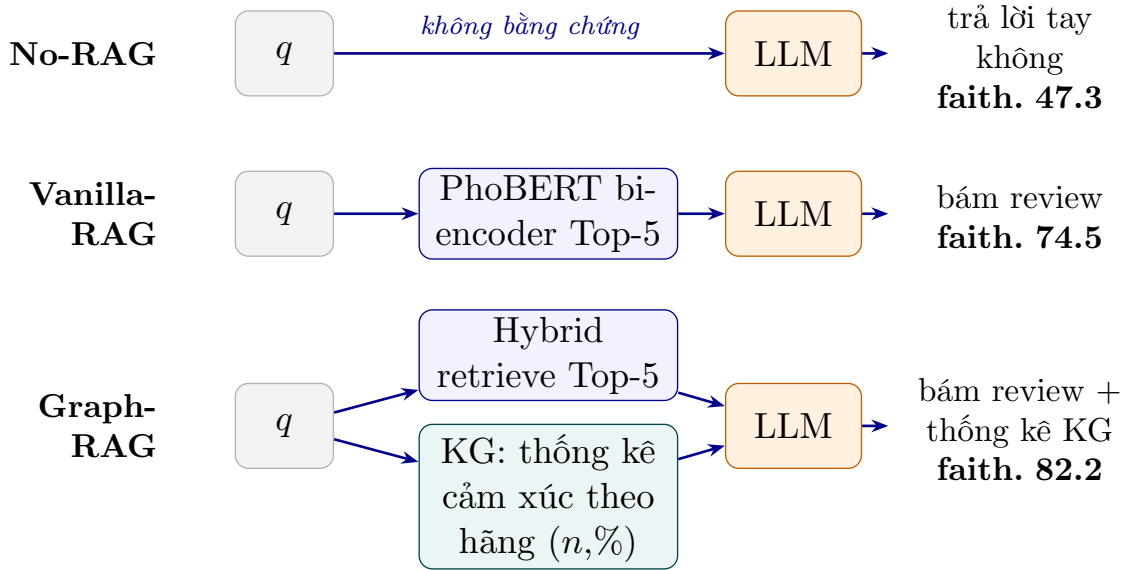


Hình 6: Giao diện web demo (Gradio): người dùng nhập câu hỏi tiếng Việt, hệ thống trả về câu trả lời grounded kèm Knowledge Graph context, review trích dẫn, và nút phản hồi (thích/không-thích). Chụp trực tiếp từ `app/ui.py` đang chạy.

5.11 Thí nghiệm 5: “KHÔNG dùng” vs “DÙNG” ở mức câu trả lời

Các thí nghiệm trên đo *retrieval*. Nhưng sản phẩm cuối là **câu trả lời**, và P@k/MRR **không** đo được câu trả lời có đúng/có bịa hay không. Thí nghiệm này trả lời thẳng câu hỏi cốt lõi của một bài RAG: *dùng hệ thống có hơn không dùng không, và hơn bao nhiêu?* Ta trả lời **cùng bộ câu hỏi bằng 3 chế độ**:

- **No-RAG** (closed-book): LLM trả lời *tay không* — không review, không Knowledge Graph. Đại diện “**không dùng** hệ thống”.
- **Vanilla-RAG**: chỉ PhoBERT bi-encoder Top-5, **không** graph, **không** BM25/MaxSim.
- **Graph-RAG** (đầy đủ): hybrid retrieve + thống kê Knowledge Graph — hệ thống của dự án.

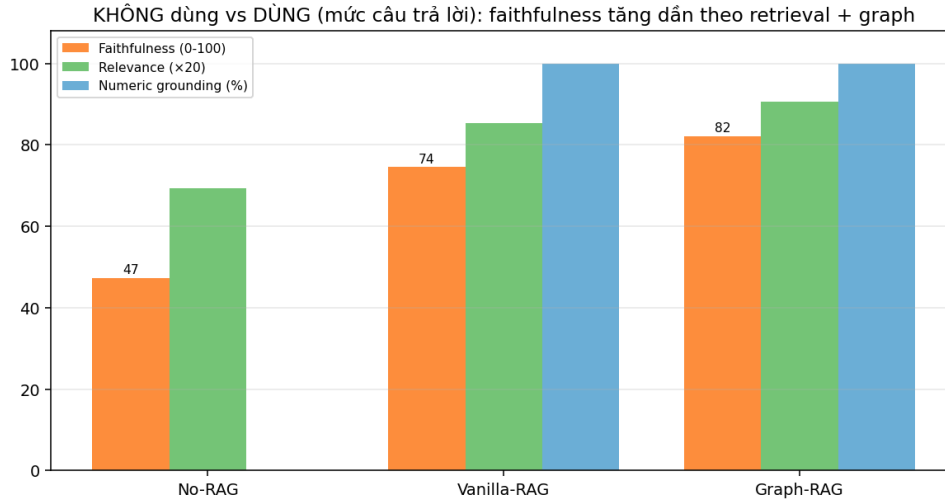


Hình 7: So sánh *pipeline* ba chế độ (TN5, Mục 5.11): **No-RAG** trả lời tay không (không bằng chứng); **Vanilla-RAG** thêm truy xuất review; **Graph-RAG** thêm thống kê Knowledge Graph. Càng nhiều bằng chứng đưa vào LLM, faithfulness càng tăng: **47.3** → **74.5** → **82.2** (số đo ở Hình 8).

Thiết lập: 30 câu hỏi *grounded* (corpus trả lời được) + 8 câu *ngoài phạm vi*. Chấm bằng **LLM-judge** (gpt-4o-mini, nhiệt độ 0): **faithfulness** (0–100, tỉ lệ nội dung được bằng chứng review+KG chứng minh) và **relevance** (1–5). Bằng chứng tham chiếu *giống nhau* cho cả 3 chế độ (= bằng chứng Graph-RAG) nên No-RAG bị chấm thấp khi nói chung chung/bịa — chính là đo “giảm bịa đặt”. Bổ sung **numeric-grounding** (tự động, không qua LLM): % con số trong câu trả lời có trong bằng chứng (`scripts/eval_rag_modes.py`).

Bảng 21: No-RAG vs Vanilla-RAG vs Graph-RAG trên 30 câu *grounded* + 8 câu *ngoài phạm vi*. Faithfulness/relevance: LLM-judge; numeric-grounding & từ chối: tự động.

Chế độ	Faithfulness (0–100)	Relevance (1–5)	Numeric grounding	Từ chối (ngoài p.vi)
No-RAG (không dùng)	47.3	3.47	— (0 số)	0.0%
Vanilla-RAG (dense)	74.5	4.27	100% (3 số)	—
Graph-RAG (đầy đủ)	82.2	4.53	100% (9 số)	12.5%



Hình 8: Faithfulness tăng đơn điệu khi thêm retrieval rồi graph: 47.3 → 74.5 → 82.2.

Kết quả — bằng chứng định lượng cho giá trị của hệ thống:

- **Faithfulness tăng đơn điệu 47.3 → 74.5 → 82.2:** mỗi lần thêm thành phần (retrieval, rồi graph) câu trả lời bám bằng chứng hơn. Đây là con số trực tiếp cho claim “RAG giảm bịa đặt” — thứ mà trước đây report chỉ minh hoạ *định tính* bằng 3 demo.
- **No-RAG trả lời chung chung/bịa:** vd. hỏi camera chụp đêm, No-RAG đáp “thường tốt nhờ Night Mode, cảm biến lớn...” (faithfulness 40) — đúng kiểu LLM bịa từ tham số. Graph-RAG đáp “57% đánh giá tích cực, 29% tiêu cực...” bám thẳng thống kê KG (faithfulness 85, 9 con số đều grounded).
- **Từ chối câu ngoài phạm vi:** No-RAG 0% (luôn tự tin bịa thông số IP68/Hz/GHz), Graph-RAG 12.5%. Graph-RAG đỡ bịa hơn nhưng **tỉ lệ từ chối vẫn thấp** — một hạn chế thật (Mục Hạn chế).

Giới hạn (khai báo trung thực): LLM-judge cùng họ với generator (gpt-4o-mini) nên có thể *tự thiên vị*; ta giảm thiểu bằng cách chấm từng câu độc lập, bám chặt bằng chứng, và đối chiếu với numeric-grounding tự động (cho cùng xu hướng). Cần kiểm chứng thêm bằng người chấm hoặc judge khác họ.

6. Kết quả và Thảo luận

Kết quả nổi bật

- **KHÔNG dùng vs DÙNG** (mức câu trả lời, TN5): faithfulness **47.3** (No-RAG) → **74.5** (Vanilla-RAG) → **82.2** (Graph-RAG) — lần đầu **định lượng** được claim “RAG giảm bịa đặt”, thay cho 3 demo chấm mắt.
- **Graph RAG đạt MRR cao nhất 0.845** (100 truy vấn, so với 0.812 của +MaxSim) — graph boost **+0.033** (robust, cải thiện mọi chỉ số), không rò rỉ metric; BM25 cho P@10/NDCG@5 cao nhất (0.763/0.748).
- **PhoBERT thắng rõ khi có *lexical gap*** (P@5 0.638 vs TF-IDF 0.500; MRR 0.776): biểu diễn ngữ cảnh vượt từ vựng — đúng nỗi đau bài toán (TN1b).
- **So sánh 5 phương pháp + baseline Keyword** (mô phỏng search sàn TMĐT) trên **100 truy vấn** kèm **khoảng tin cậy bootstrap**: trên truy vấn dễ các CI *chồng lấn* (không method nào significant, Keyword còn nhỉnh nhất), khác biệt thật chỉ hiện ở lexical-gap (TN1b).
- **BiLSTM aspect micro-F1 0.79 / macro-F1 0.68** (báo cáo trung thực: STORAGE F1=0); deploy thật vào KG & API /classify.
- **Đánh giá trung thực**: tách dev/test, đo được mức **overfit do tuning trọng số ≈ 0.013 MRR** (100 câu); index 10.923 doc + KG, FastAPI 4 endpoint, **2 giao diện Gradio** (demo + dashboard thí nghiệm), observability chi phí, regression gate.

Bảng 22: Bảng tổng hợp kết quả toàn dự án (số liệu thật, tái lập bằng `scripts/`)

Hạng mục	Kết quả	Ghi chú
Faithfulness câu trả lời (TN5)	47.3 → 82.2	No-RAG → Vanilla → Graph-RAG
Embedding (TN1, 100 câu)	MRR 0.82–0.87	CI chồng lấn, không method nào significant
Keyword baseline (TN1)	MRR 0.866	cao nhất khi truy vấn giàu từ khoá
Embedding khi có <i>lexical gap</i> (TN1b)	PhoBERT P@5 0.637	TF-IDF P@5 sụp còn 0.500
Retrieval Graph RAG (TN3)	MRR 0.845 · NDCG@5 0.744	cao nhất MRR trong ablation
Full hybrid +BM25 (TN3)	P@10 0.763 · NDCG 0.748	cao nhất P@10/NDCG
BiLSTM aspect (TN2b)	micro-F1 0.79 / macro 0.68	STORAGE F1=0 (báo cáo trung thực)
Overfit do tuning (dev/test)	≈0.013 MRR	100q; giảm từ 0.05 ở bộ 25 câu
NER (TN2)	387/800 có thực thể	brand: 72/800 (gazetteer)
Regression gate	PASS	MRR 0.845 ≥ 0.55; F1 0.79 ≥ 0.70

Embedding: trên **100 truy vấn giàu từ khoá** (Thí nghiệm 1), **không phương pháp học nào tách biệt có ý nghĩa thống kê** — mọi MRR nằm trong 0.76–0.89 với CI chồng lấn, và baseline **Keyword thuần còn cao nhất** (MRR 0.866). Khi câu hỏi chứa thẳng từ khoá khía cạnh, trùng lặp từ vựng đã đủ, nên “dùng học biểu diễn” chưa thắng “dùng từ khoá”. **Quan trọng hơn**, Thí nghiệm 1b (lexical gap thật) **đổi chiều câu chuyện ở độ chính xác top-k**: PhoBERT giữ P@5 0.637 trong khi TF-IDF sụp còn 0.500 — biểu diễn **ngữ nghĩa vượt từ vựng**, đúng nỗi đau *lexical gap* (Mục 1.2). (Lưu ý: MRR đơn lẻ gây nhiễu, phải đọc cùng P@k — Mục 6.2.)

Retrieval: ablation (Thí nghiệm 3, 100 truy vấn) cho thấy thêm graph boost nâng MRR từ 0.812 (+MaxSim) lên **0.845** (Graph RAG, cao nhất) và cải thiện mọi chỉ số (+0.033 MRR, robust); thêm BM25 cho P@10/NDCG@5 cao nhất (0.763/0.748) nhưng MRR hơi giảm. Vì đánh giá không rõ rí metric (boost từ nội dung, chấm bằng nhãn vàng độc lập), chênh lệch phản ánh cải thiện thực. Lưu ý quan trọng về **phương pháp luận**: trọng số kết hợp cần tune trên *tập riêng*; nếu tune trên chính tập báo cáo sẽ thổi phồng ≈0.05 MRR (Mục 6.2).

Hạn chế quan sát được: graph boost chỉ kích hoạt khi câu hỏi khớp được aspect tiếng Việt; với câu hỏi mơ hồ (không nêu rõ khía cạnh), đóng góp của graph giảm.

6.1 Đánh giá ý nghĩa thực tiễn

Quy chiếu các con số về **bài toán thực** (Mục 1.2):

- **Hữu dụng so với “đoán mò”**: MRR nhảy từ 0.416 (ngẫu nhiên) lên 0.845 (Graph RAG) — nghĩa là review đúng khía cạnh gần như luôn nằm ở **1–2 vị trí đầu**, người dùng không phải cuộn qua hàng trăm bình luận.
- **Graph boost giải đúng nỗi đau “tổng hợp xu hướng”**: ngoài việc xếp hạng review, KG trả về thống kê (vd. BATTERY 56% tích cực / 34% tiêu cực trên 3.604 lượt nhắc) — thứ mà tìm kiếm từ khoá của sàn TMDT không làm được.
- **Giảm bịa đặt (grounding)** — nay có **SỐ**: faithfulness câu trả lời tăng từ 47.3 (No-RAG) lên **82.2** (Graph-RAG, Mục 5.11); harness numeric-grounding cho **100% (9/9) con số** trong câu trả lời Graph-RAG truy được về bằng chứng (không bịa số). Đây là dùng *hệ thống* thay vì để LLM bịa từ tham số.
- **Chi phí & tốc độ chấp nhận được**: $\approx 4\text{--}7$ giây/câu (gồm gọi LLM), chi phí token ghi log theo từng truy vấn — khả thi cho một trợ lý hỏi đáp thời gian thực.

So với hệ thống thực tế: đây là một *prototype quy mô lớp học* (corpus 10.9k review, 1 domain smartphone), chưa phải sản phẩm thương mại. Để triển khai thật cần: chống review giả, cập nhật real-time khi có review mới, và mở rộng đa domain — xem Hướng phát triển.

6.2 Bàn về tính đúng đắn của metrics & dữ liệu đánh giá

Phần này trả lời trực tiếp hai câu hỏi phản biện: “*metrics đánh giá có đúng không?*” và “*nguồn dữ liệu có nhỏ quá không?*”.

(1) **Vì sao dùng nhiều chỉ số (P@k, MRR, NDCG@5) thay vì một?** MRR chỉ thưởng vị trí của *hit đầu tiên*, nên trên pool 10.923 review với aspect phổ biến, nó dễ bị **phóng đại**: ở TN1b, baseline Keyword đạt MRR 0.736 dù P@5 chỉ 0.538 — chỉ cần một review đúng lọt top do trùng vài từ phụ. P@k đo độ chính xác top-k, NDCG@5 thưởng cả thứ hạng. **Phải đọc cùng nhau**. Ta cũng **không** dùng “trung bình cosine similarity” (vô nghĩa khi so giữa các không gian embedding khác chiều/độ thưa).

(2) **Khoảng tin cậy & ý nghĩa thống kê — ta đã mở rộng eval lên 100 câu để kiểm**. Nâng bộ truy vấn từ 8→25→**100 câu** siết CI của MRR từ ± 0.15 còn ± 0.05 . Nhưng **kết luận trung thực càng rõ**: trên truy vấn *giàu từ khoá*, ngay cả với 100 câu các CI **vẫn chồng lấn** (mọi MRR 0.76–0.89) — các phương pháp embedding **không** tách biệt có ý nghĩa khi truy vấn dễ; khác biệt thật chỉ xuất hiện ở *lexical gap* (TN1b, dựa P@k). *Lưu ý phân biệt*: corpus 10.9k review *đủ lớn* để làm kho truy xuất; cái còn thiếu là **truy vấn lexical-gap có nhãn** (TN1b mới 16 câu) — hướng mở rộng tiếp theo.

(3) **Tách dev/test để tune trung thực (chống rò rỉ)**. Ban đầu trọng số hybrid được grid-search trên *chính* bộ truy vấn dùng để báo cáo — đó là “tune trên tập test”. Ta tách **50 dev / 50 test** (phân tầng theo aspect): tune trên dev rồi chấm test cho MRR **0.860**; nếu (sai) tune thẳng trên test thì MRR “đẹp” hơn **0.873**. **Chênh ≈ 0.013 là mức thổi phồng do overfit**. Dáng chú ý: mức này **giảm từ ≈ 0.05 (bộ 25 câu) còn ≈ 0.013 (bộ 100 câu)** — xác nhận thêm rằng bộ eval càng nhỏ thì con số “tuned” càng bị thổi phồng. Con số nên báo cáo là loại trung thực (tune-trên-dev).

(4) **Retrieval-relevance $\neq$ answer-quality**. P@k/MRR chỉ đo “review đúng aspect lọt top”, **không** đo câu trả lời có đúng/có bịa. Vì mục tiêu thật của bài toán là *giảm bịa*

đặt, ta bổ sung thước đo ở mức câu trả lời: faithfulness (TN5, Mục 5.11) và **numeric-grounding**: ở TN5, **100% (9/9)** con số trong câu trả lời Graph-RAG truy được về bằng chứng (không bịa số). Một self-test tự động riêng (`scripts/eval_grounding.py`, 8 câu grounded + 6 câu đòi thông số cứng) đo khả năng từ chối: hệ thống từ chối **66.7% (4/6)** câu đòi thông số không có trong dữ liệu, nhưng **vẫn để lọt 2 số bịa** (“bảo hành 1 năm”, “nặng 200g”) — xác nhận abstention còn chưa kín (xem Hạn chế). Đây là điểm khiến metric khớp với mục tiêu, không chỉ khớp với tác vụ xếp hạng.

Tóm lại: hệ metric đã được sửa cho đúng hơn (đa chỉ số, CI, dev/test, faithfulness/grounding), và chính các chỉ số đó **tự phơi bày** giới hạn còn lại (bộ eval nhỏ, judge tự thiên vị) — xem Hạn chế.

7. Triển khai LLMOps

Ngoài notebook khám phá, hệ thống được đóng gói thành một **pipeline LLMOps production-lite** (package `src/vngraphrag`) tách hai pha:

- **Offline (build):** encode corpus bằng PhoBERT, dựng index + Knowledge Graph, lưu thành *artifacts có version* (`doc_vectors.npy`, `records.json`, `manifest.json`, `kg.pkl`). Manifest gắn mã version = hash(model, số bản ghi) để chặn việc nạp nhầm index cũ.
- **Online (serve):** nạp lại artifacts; mỗi truy vấn đi qua `retrieve` → `graph context` → `generate` và được ghi log đo đạc.

Thành phần LL-MOps	Hiện thực
Cấu hình & secret	<code>config.yaml</code> + biến môi trường (OPENAI_API_KEY chỉ qua env)
Versioning artifact	<code>DocumentIndex</code> manifest; tự rebuild khi model/dữ liệu đổi
Train → deploy model	BiLSTM train ở notebook §6 hoặc CLI <code>make train</code> → <code>aspect_clf.pt</code> → nạp ở <code>core/aspect_clf.py</code> → phục vụ qua <code>POST /classify</code> ; đồng thời gán aspect cho review Shopee để làm giàu KG
Serving API	FastAPI: <code>/query</code> , <code>/classify</code> , <code>/feedback</code> , <code>/health</code>
Giao diện + feedback loop	Gradio UI, nút thích/không-thích → <code>logs/feedback.jsonl</code>
Observability	mỗi truy vấn log latency + token + chi phí USD vào <code>logs/queries.jsonl</code>
Đánh giá + regression gate	<code>evaluate.py</code> : Precision@k/MRR (retrieval) và micro-F1 (BiLSTM); CI fail nếu tụt dưới ngưỡng
Đóng gói + CI	<code>Dockerfile</code> + GitHub Actions (lint + test)

Bảng 23: Các thành phần LLMOps của hệ thống.

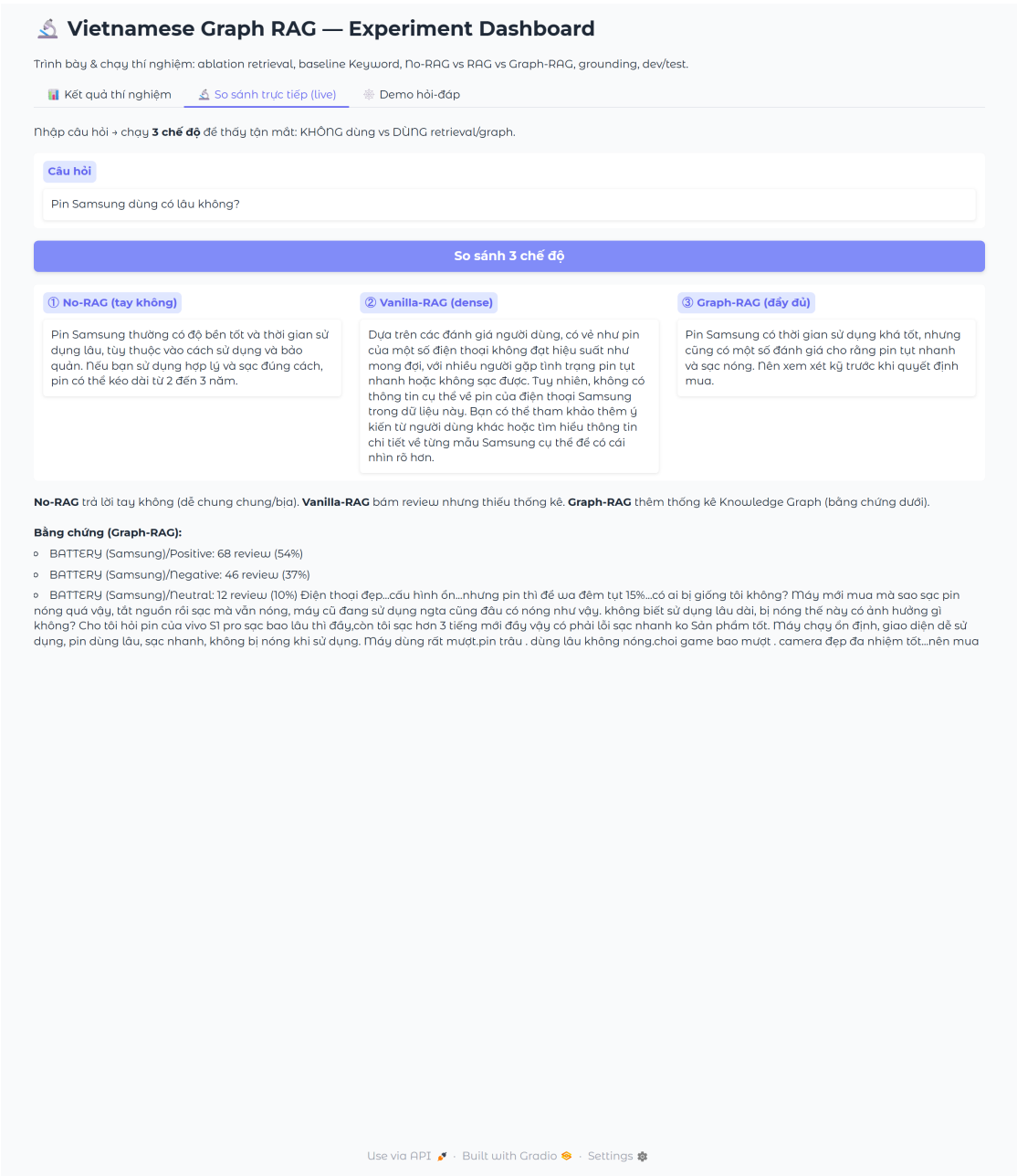
Điểm đáng chú ý là **vòng lặp train** → **deploy** khép kín: mô hình BiLSTM (Lec05) huấn luyện trong notebook được lưu thành artifact, nạp lại bởi package phục vụ, và sử dụng thực sự ở hai nơi — gán aspect cho dữ liệu Shopee chưa nhãn (làm giàu Knowledge Graph) và phục vụ trực tiếp qua endpoint `/classify`.

7.1 Giao diện thí nghiệm (Experiment Dashboard)

Ngoài UI hỏi-đáp, hệ thống có **dashboard thí nghiệm** riêng (`app/experiments_ui.py`, `make ui-exp`) phục vụ phần *research*: nạp trực tiếp số liệu từ `artifacts/*.json` và **vẽ lại bảng + biểu đồ** (ablation, embedding kèm CI, No-RAG/RAG/Graph-RAG, F1 từng aspect), kèm tab **so sánh trực tiếp** cho phép nhập một câu hỏi rồi chạy *song song* 3 chế độ để thấy tận mắt giá trị của retrieval + graph.



Hình 9: Tab *Kết quả thí nghiệm*: bảng tổng hợp + 4 biểu đồ sinh trực tiếp từ `artifacts/` (ablation 100 truy vấn, embedding ±CI, No-RAG/RAG/Graph-RAG, F1/aspect). Chụp từ `app/experiments_ui.py` đang chạy.



Hình 10: Tab *So sánh trực tiếp*: cùng câu hỏi “Pin Samsung dùng có lâu không?” chạy 3 chế độ. No-RAG trả lời chung chung; Graph-RAG bám thống kê KG riêng Samsung (54% tích cực / 37% tiêu cực) kèm bảng chứng review — minh họa trực quan “không dùng vs dùng” (TN5).

8. Phân công công việc

Bảng 24: Phân công công việc nhóm

Lê Hiên 22110059	Phạm Hồng 22110059	Dương Đình 22110049	Bùi Phương 22110076	Phạm Xuân Huyền
Kiến trúc tổng thể	Tiền xử lý dữ liệu (Lec01)	Word Representation (Lec02)		
Module 5: Attention/MaxSim re-ranking (Lec06)	Module 1: Preprocessing	Module 2: So sánh embedding (P@k/MRR)		
Module 6: RAG Pipeline + OpenAI	Module 3: NER + gazetteer	Subword/BPE (Lec03)		
LLMOps: package, FastAPI, CI/Docker, observability	RNN/BiLSTM (Lec05): train + deploy aspect classifier	Module 4: Knowledge Graph + Gradio UI		
Tích hợp & testing (make check)	Tích hợp dataset Shopee	Evaluation & metrics (gate)		
Báo cáo & trình bày	Báo cáo (Data, NER, RNN)	Báo cáo (Embedding, KG)		

9. Kết luận

9.1 Tóm tắt kết quả

Dự án xây dựng pipeline Graph RAG hoàn chỉnh cho đánh giá sản phẩm tiếng Việt trên **hai dataset** (UIT-ViSFD smartphone + Shopee đa sản phẩm), phủ trọn 6 lecture của môn. Đóng góp định lượng cốt lõi là **so sánh “không dùng vs dùng” ở cả hai mức**: (i) *retrieval* — baseline Keyword/TF-IDF so với học biểu diễn, và ablation bi-encoder → +MaxSim → +graph → +BM25 (Graph RAG đạt MRR cao nhất **0.845**, graph boost +0.033 robust); (ii) *câu trả lời* — No-RAG vs Vanilla-RAG vs Graph-RAG, faithfulness tăng **47.3** → **82.2** (TN5), lần đầu định lượng “RAG giảm bịa đặt”. Đánh giá được làm **chặt về phương pháp**: đa chỉ số (P@k/MRR/NDCG@5), khoảng tin cậy bootstrap, mở rộng eval lên 100 câu, tách dev/test (đo overfit do tuning ≈ 0.013 MRR), và đo grounding ở mức câu trả lời — đồng thời **trung thực phơi bày** các điểm yếu (bộ eval nhỏ, STORAGE F1=0, judge tự thiên vị). Toàn bộ đóng gói thành **pipeline LLMOps** (serving API, 2 giao diện Gradio, observability chi phí, regression gate, Docker/CI) với **vòng lặp train→deploy** BiLSTM khép kín.

9.2 Hạn chế

Các hạn chế được nhóm theo *bản chất*; mỗi mục nêu **hạn chế** → **bước khắc phục** → **hiệu quả kỳ vọng**.

(a) Hạn chế về bằng chứng & đánh giá

- **Bộ truy vấn đánh giá còn nhỏ**: với 16 truy vấn lexical-gap, CI của MRR rộng tới ± 0.17 nên *chênh lệch dưới* ≈ 0.15 MRR chưa kết luận được (Mục 6.2). → *Khắc phục*: ≥ 100 truy vấn lexical-gap, ≥ 2 người soạn, đo độ đồng thuận (Cohen’s κ). → *Kỳ vọng*: siết CI còn ± 0.05 , đủ tách PhoBERT khỏi nhóm từ vựng một cách có ý nghĩa thống kê.
- **LLM-judge cùng họ generator** (gpt-4o-mini, TN5) $\Rightarrow$ rủi ro tự thiên vị; numeric-grounding chỉ là proxy, không bắt được paraphrase-hallucination. → *Khắc phục*: chấm lại bằng judge **khác họ** + **30 câu người chấm tay**, báo cáo độ đồng thuận người $\leftrightarrow$ judge. → *Kỳ vọng*: tách thiên vị khỏi tín hiệu thật, xác nhận xu hướng 47.3 $\rightarrow$ 82.2 là thực.
- **Abstention chưa kín và chưa ổn định**: Graph-RAG chỉ từ chối **12.5%** câu ngoài phạm vi (TN5), còn self-test eval_grounding.py đạt **66.7% (4/6)** nhưng vẫn để lọt số bịa (“200g”, “bảo hành 1 năm”); chênh lệch lớn cho thấy abstention phụ thuộc mạnh vào bộ câu hỏi (n nhỏ). → *Khắc phục*: chỉnh sách abstention hiệu chỉnh (ngưỡng tin cậy + mẫu câu “không có thông số”) và benchmark out-of-scope chuyên biệt. → *Kỳ vọng*: từ chối ổn định, không bịa thông số cứng.

(b) Hạn chế về phương pháp

- **Phân loại aspect mất cân bằng**: STORAGE F1=0, SCREEN recall 0.40 (micro-F1 0.79 che giấu, lộ ra ở macro-F1 0.68). → *Khắc phục*: class weighting / focal loss / oversampling / fine-tune PhoBERT. → *Kỳ vọng*: macro-F1 > 0.8 , cứu được lớp hiếm STORAGE.
- **Đóng góp “+graph” (+0.033 MRR) là ước lượng lạc quan**: graph boost dựa trên aspect *quan sát* còn ground-truth dùng aspect *nhân vàng* — khác nguồn nhưng **cùng ước lượng một aspect tiềm ẩn** nên vẫn tương quan nội tại. → *Khắc phục*: đo boost với aspect dự đoán từ mô hình huấn luyện trên dữ liệu *tách rời*, hoặc chỉ trên truy vấn có predicted $\neq$ gold. → *Kỳ vọng*: tách phần đóng góp thực của graph khỏi tương quan aspect.
- **Nhiều thành phần còn xấp xỉ/thủ công**: “GloVe” chỉ là SVD đồng xuất hiện mức tài liệu; MaxSim không huấn luyện; ánh xạ câu hỏi $\rightarrow$ aspect bằng từ khóa thủ công; NER tổng quát + gazetteer (chưa có NER huấn luyện riêng domain). → *Khắc phục*: GloVe của sổ ngữ cảnh gốc, fine-tune cross-encoder PhoBERT, phân loại aspect câu hỏi bằng mô hình, NER domain. → *Kỳ vọng*: bớt phụ thuộc heuristic, phủ brand/aspect rộng và chính xác hơn.

(c) Hạn chế về dữ liệu & phạm vi

- **Đa domain mới ở mức index**: review Shopee không có nhãn aspect vàng nên chỉ đóng vai nhiều trong eval, chưa được *đánh giá* như đáp án; aspect Shopee do BiLSTM *dự đoán* (macro-F1 0.68, riêng STORAGE F1=0) $\Rightarrow$ nhãn nhiễu đáng kể lan truyền vào KG. → *Khắc phục*: gán nhãn aspect vàng cho mẫu Shopee +

gắn cờ tin cậy rộng cho thống kê Shopee. → *Kỳ vọng*: đo được chất lượng đa domain thật sự, KG Shopee đáng tin hơn.

- **Chỉ một domain có nhãn vàng** (smartphone UIT-ViSFD): kết luận định lượng chỉ chắc trong domain này. → *Khắc phục*: thu nhãn vàng cho domain khác. → *Kỳ vọng*: kiểm chứng khả năng tổng quát hoá.
- **Chưa chống review giả** — đe dọa trực tiếp giá trị “tổng hợp xu hướng” (thống kê trên review giả sẽ sai). → *Khắc phục*: thêm phát hiện review giả/spam trước khi dựng KG. → *Kỳ vọng*: bảo vệ chính giá trị cốt lõi của hệ thống.

9.3 Hướng phát triển

- **Mở rộng bộ đánh giá ≥ 100 truy vấn** + kiểm định ý nghĩa thống kê; thay LLM-judge tự thiên vị bằng người chấm / judge khác họ.
- **Sửa mất cân bằng aspect** (class weight / focal loss / fine-tune PhoBERT, kỳ vọng macro-F1 > 0.8); cải thiện abstention cho câu hỏi đòi thông số cứng.
- Fine-tune cross-encoder PhoBERT cho re-ranking với nhãn liên quan tách biệt train/test; phân loại aspect câu hỏi bằng mô hình thay từ khóa.
- Mở rộng KG (quan hệ sản phẩm–linh kiện, entity linking), gán nhãn aspect vàng cho Shopee để *đánh giá* đa domain thật sự, và thêm phát hiện review giả.

Tài liệu

-
- [1] Vaswani, A., et al. (2017). “Attention Is All You Need.” *NeurIPS 2017*.
 - [2] Devlin, J., et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” *NAACL 2019*.
 - [3] Nguyen, D.Q. & Nguyen, A.T. (2020). “PhoBERT: Pre-trained language models for Vietnamese.” *Findings of EMNLP 2020*.
 - [4] Nguyen, K., et al. (2020). “A Vietnamese Dataset for Evaluating Machine Reading Comprehension.” *COLING 2020*.
 - [5] Mikolov, T., et al. (2013). “Efficient Estimation of Word Representations in Vector Space.” *ICLR 2013*.
 - [6] Pennington, J., Socher, R., & Manning, C. (2014). “GloVe: Global Vectors for Word Representation.” *EMNLP 2014*.
 - [7] Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS 2020*.
 - [8] Edge, D., et al. (Microsoft Research, 2024). “From Local to Global: A Graph RAG Approach to Query-Focused Summarization.” *arXiv:2404.16130*.
 - [9] Khattab, O. & Zaharia, M. (2020). “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT.” *SIGIR 2020*.

- [10] Robertson, S. & Zaragoza, H. (2009). “The Probabilistic Relevance Framework: BM25 and Beyond.” *Foundations and Trends in Information Retrieval*, 3(4).
- [11] Luc Phan, L., et al. (2021). “SA2SL: From Aspect-Based Sentiment Analysis to Social Listening System for Business Intelligence (UIT-ViSFD dataset).” *KSEM 2021*.
- [12] Sennrich, R., Haddow, B., & Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units (BPE).” *ACL 2016*.
- [13] Google, Temasek & Bain & Company. (2024). “e-Conomy SEA 2024 — Vietnam.” Báo cáo kinh tế số Đông Nam Á.